WEB-BASED STUDENTS RESULT UPLOAD COMPLAINT SYSTEM

BY

UCHE INNOCENT EMMANUEL

2020/SC/12673

A PROJECT SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE AND INFORMATICS IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF BACHELOR OF SCIENCE, ALEX EKWUEME FEDERAL UNIVERSITY, NDUFU-ALIKE, IKWO, EBONYI STATE, NIGERIA.

OCTOBER 2024

DECLERATION

I UCHE INNOCENT EMMANUEL, an undergraduate student in computer science program, in the Department of computer science/informatics, with registration number:2020/SC/12673, declare that the work embedded in this research report is original and has not been submitted in full or in part for any other degree or diploma of this university or any other institution of higher earning .

_____

UCHE INNOCENT EMMANUEL

2020/SC/12673

CERTIFICATION

This is to certify that I Uche Innocent Emmanuel prepared, compiled and submitted this project with the registration number: 2020/SC/12673, under the supervision Dr. Mrs Nneka E. Richard-Nnabu to computer science programme in the department of computer

science/informatics, having been accepted on meeting part of the requirements for the awards of bachelor of science.

_____ _____

Dr. Mrs. N. E. Richard-Nnabu Date

(Project Supervisor)

_____ _____

Dr. I.I Achi Date

(Head of Department)

APPROVAL

This is to certify that this research titled WEB BASED STUDENTS RESULT UPLOAD COMPLAINTS SYSTEM, was carried out by (UCHE INNOCENT EMMANUEL with the registration number:2020/SC/12673) under our supervision in the Department of computer science/informatics Alex Ekwueme Federal University Ndufu-Alike, Ebonyi State Nigeria.

_____ _____

DR. Mrs. N.E. Richard-Nnabu Date (Project Supervisor)

_____ _____

Dr. Mrs. Doris Nnakwuzie Date

(Project Coordinator)

_____ _____

DR. I.I Achi Date

(Head of Department)

_____ _____

Dr. A. N. Nwachuwu Date

(Dean)

_____ _____

Prof.R.O.Okonkwo Date

(External Examiner)

DEDICATION

This project is dedicated to my parents, Mr & Mrs Innocent Nwode, whose unwavering love, support and sacrifices have shaped my life and aspirations. Your belief in me has been my greatest motivation.

ACKNOLEDGEMENT

I am immensely grateful to all those who contributed to the success of this project.

First and foremost, I extend my heartfelt appreciation to my supervisor, DR. MRS NNEKA E. RICHARD-NNABU, for her expert guidance, invaluable insights, and continuous support throughout every phase of this project. Her mentorship and encouragement have been instrumental in shaping this work.

I am deeply grateful to our program coordinator, Dr. Mrs. Doris Nnakwuezie, for her steadfast support and guidance. Her encouragement and commitment to academic excellence have provided me with the resources and motivation to reach my goals.

My sincere thanks also go to the Head of Department, Dr. I.I. Achi, for his inspiring leadership and for creating an environment that encourages research and innovation. His support for student-led projects has been a significant driving force in my academic journey.

I would also like to acknowledge the entire faculty, whose instruction and insights have contributed to my growth and learning, laying the foundational knowledge and skills that made this project possible.

A special note of thanks goes to my family and friends, whose support, patience, and encouragement have been invaluable. Their belief in my potential has provided me with the strength to persevere. I am equally grateful to my data sources and all those who provided critical information and resources, enabling this project to be rooted in real-world insights.

To all, thank you for your contributions, guidance, and unwavering support. This accomplishment would not have been possible without you.

ABSTRACT

The management and resolution of student complaints related to result uploads is a critical task in educational institutions, directly impacting student satisfaction and academic transparency. Many institutions still rely on manual processes for handling complaints, which can lead to inefficiencies, delays, and inaccuracies in complaint resolution. To address this, we propose a Web-Based Student Result Upload Complaint System, a solution designed to automate and streamline the complaint submission, tracking, and resolution process. The proposed system provides students with an online platform to log result upload complaints, while allowing administrative staff to effectively manage, track, and resolve these complaints. This web-based system was developed using a hybrid methodology that combines Agile and Object-Oriented Analysis and Design (OOAD) principles, offering a flexible, iterative approach to system development and a structured framework for the system's architecture. Implemented using Node.js, Express.js, MongoDB, and front-end technologies including HTML, CSS, JavaScript, and jQuery, the system offers a user-friendly interface for students and administrators, ensuring seamless interactions and efficient data processing. To enhance functionality, the system includes features for complaint categorization, priority assignment, status updates, and automated notifications. The system's architecture was designed to support scalability, ensuring it can accommodate large numbers of users and complaints. Testing results demonstrate the system's effectiveness in reducing complaint resolution times, improving accuracy, and enhancing the overall management of result upload issues.

Table of Contents

Cover Page

Title i

Declaration ii

Certification iii

Approval iv

Dedication v

Acknowledgements vi

Abstract vii

Chapter One: Introduction 1

1.1 Background of the Study 1

1.2 Statement of the Problem 2

1.3 Aim and Objectives of the Study 3

1.4 Significance of the Study 4

1.5 Scope of the Study 5

1.6 Limitation of the Study 5

1.7 Definition of Terms 5

Chapter Two: Literature Review 6

2.1 Introduction 6

2.2 Theoretical Background 9

2.2.1 Conceptual Terms 11

2.2.2 Technology and Tools for Implementation 11

2.3 Review of Related Literature 12

2.4 Research Gap/Literature Gap 14

CHAPTER THREE: RESEARCH METHODOLOGY, SYSTEM ANALYSIS AND DESIGN 15

3.1 Methodology 15

3.2 System Analysis 16

3.2.1 Data Gathering Technique 22

3.2.2 Analysis of the Existing System 24

3.2.3 Advantages of the Existing System 27

3.2.4 Disadvantages of the Existing System 27

3.2.5 High Level Model of the Proposed System 28

3.2.6Analysis of the Proposed System 29

3.2.7 Justification of the Proposed System 32

3.3 System Design 35

3.3.1 Objective of the Design 36

3.3.2 Algorithm 37

3.3.3 System Architecture 39

3.3.4 Main Menu Design 41

3.3.5 Sub Menu Design 41

3.3.5.1/3.3.5.2 Pick specific Sub System and Design 42

3.3.6 Program Module Design (Pick at most 2 and design) 43

3.3.7 Database Development Tool 44

3.3.8 Data Dictionary 45

3.3.9 Database Design and Structure 47

3.3.10 Mathematical Specification (if any) 48

3.3.11 Program Module Specification (if any) 49

3.3.12 Input / Output Format 50

Chapter Four: System Implementation, Testing and Results 52

4.1 System Implementation 52

4.1.1 Hardware Requirement 53

4.1.2 Software Requirement 55

4.1.3 Program Development 55

4.1.3.1 Choice of Programming Language 57

4.1.3.2 Language Justification 58

4.1.4 Main Menu Implementation 61

4.1.5 Sub Menu Implementation 62

4.1.6 Program Module Implementation 63

4.2 System Testing 65

4.2.1 Test Plan 66

4.2.2 Test Data 66

4.3. Results 69

4.3.1 Actual Test Result versus Expected Test Result 71

4.3.2 Performance Evaluation 73

4.3.3 Limitation of the Result 73

4.3.4 Results and Discussions 74

4.4 System Security (if any) 75

4.5 System Integration 76

4.6 Documentation (Optional) 76

Chapter Five: Summary, Conclusion and Recommendations **78**

5.1 Summary 78

5.2 Conclusion 78

5.3 Recommendations 79

5.4 Contributions to Knowledge 79

References 81

Appendix A – Source Code 87

Appendix B – Sample Output 92

LIST OF TABLE

4.1Actual Test Result versus Expected Test Result 72 **LIST OF FIGURE**

3.1 Use Case Diagram 20

3.2 System Data Flow Diagram 21

3.2.5High Level Model of the Proposed System 29

3.3 showing the System UML 36

3.3.2: System algorithm using Flow chart 38

3.3.3 System Architecture 40

3.3.4 show Main menu 41

3.3.5 Sub Menu Design 42

3.3.5: show User management Sub menu. 42

3.3.6: Registration Module Using Flowchart. 43

3.3.7: Login Module Using Flowchart. 44

3.3.9 show the student collection in mongoose schema 45

3.3.10 show result collection in mongoose schema 46

3.3.11 show staffs & admin collection in mongoose schema 46

3.3.9 show complaint collection in mongoose schema 47

3.3.9 show complaint collection in mongoose schema 47

3.3.9 show the student collection in mongoose schema 48

3.3.11 show staffs & admin collection in mongoose schema 49

3.3.12: Program Module Of the system 50

3.3.12 Input / Output Format 51

4.1:Tools and Technologies 53

4.2: Main Menu Design. 62

4.3: Student Dashboard on a Successful Login. 63

4.5:Staff Dashboard for Uploading Student Result 64

4.6:Admin dashboard viewing students records 64

4.7: Staff Dashboard for Updating of Result Complaints 65

4.8: Test Data (User Valid Data)JSON format 67

4.9: show codes for validating invalid data(Regex expression) 67

4.10: Codes for boundary data validation (file upload eg image) 68

4.3.1: Successful login 70

4.3.2: Complaint submission module 71

CHAPTER ONE

INTRODUCTION

1.1 **Background of the Study**

In contemporary educational institutions, the management and dissemination of student academic results play a pivotal role in shaping the academic journey and experience of learners. According

to (Smith *et al.,* 2018), timely and accurate delivery of these results not only reflects the institution's commitment to academic excellence but also significantly impacts student satisfaction and performance. However, despite rigorous efforts to maintain efficiency in result management systems, challenges such as errors in result processing, delays in result publication, and discrepancies in grading can arise, leading to frustration and dissatisfaction among students.

In response to these challenges, the development of a web-based student results upload complaint system emerges as a crucial endeavor (Brown et al., 2016). This system aims to address the shortcomings of traditional result management processes by providing a transparent, efficient, and user-friendly platform for students to report discrepancies or grievances related to their academic results. By leveraging the power of web technologies, the proposed system seeks to streamline the complaint submission process, enhance communication between students and academic authorities, and ultimately foster a culture of accountability and transparency within educational institutions.

The significance of implementing such a system extends beyond the mere resolution of individual complaints. It embodies a proactive approach towards enhancing the overall quality of academic administration and student services. By facilitating prompt resolution of resultrelated issues, the system contributes to maintaining the integrity of academic assessments, safeguarding the rights of students, and upholding the reputation of the institution. Moreover, it serves as a testament to the

institution's commitment to continuous improvement and responsiveness to the evolving needs of its student community.

As educational institutions embrace digital transformation and strive to adapt to the demands of an increasingly interconnected world, the importance of robust and responsive result management systems cannot be overstated. Through this study, we aim to explore the design,

development, and implementation of a web-based student results upload complaint system, examining its efficacy in addressing student grievances, enhancing administrative efficiency, and fostering a culture of transparency and accountability in academic affairs(Garcia, *et al,* 2018). By elucidating the benefits and challenges associated with such a system, this research seeks to offer valuable insights and recommendations for educational stakeholders seeking to optimize their result management" processes and elevate the overall student experience.

1.2 Statement of the Problem

I. **Inefficiencies in traditional result management systems:** Traditional methods of managing student results often lead to delays, errors, and discrepancies in result processing and publication, causing frustration and dissatisfaction among students (Wang & Huang., 2021).

II. **Lack of transparent communication channels:** Existing communication channels between students and academic authorities for reporting result-related grievances are often opaque and cumbersome, hindering the timely resolution of complaints and fostering a sense of mistrust in the result management process (Li & Tan., 2017).

III. **Need for absence of streamlined complaint resolution process:** There is a pressing need for the development of a streamlined and user-friendly web-based platform that enables students to submit, track, and receive timely resolution of complaints regarding their academic results, thereby enhancing transparency, accountability, and student satisfaction (Chen & Wu., 2019).

IV. **Security and privacy concerns:** Concerns regarding the security and privacy of student information persist, particularly in the context of implementing a web-based system for result complaint submission, necessitating robust measures to safeguard sensitive data and comply with relevant data protection regulations.

V. **Integration challenges with existing systems:** The integration of a web-based student results upload complaint system with existing academic management systems poses technical and logistical challenges, including compatibility issues, data synchronization, and user adoption, which need to be addressed to ensure seamless implementation and usability.

1.3 Aim and Objectives of the Study

The Primary aim of this study is to develop a web-based complaint system that allows students to file and track issues related to result uploads, facilitating streamlined communication with the institution's administration for a faster resolution.

Objectives of the study include to:

i. Develop a User-Friendly Interface ii. Implement Secure Authentication and Authorization Mechanisms iii. Enable Results Upload Functionality iv. Facilitate Complaint Submission

v.Establish a Notification System

1.4 Significance of the Study:

The significance of the proposed project topic is as follows:

I. **Enhanced Transparency:** Implementing a web-based complaint system improves transparency in academic processes. Students can easily track the progress of their complaints, view uploaded results, and receive notifications about resolution status. This transparency fosters trust between students and academic authorities.

II. **Improved Efficiency:** By streamlining the complaint submission and resolution process, the system reduces the time and effort required for both students and

administrators. Result discrepancies can be addressed promptly, leading to faster resolutions and ultimately enhancing the overall efficiency of academic administration.

III. **Empowerment of Students:** Providing students with a platform to voice their concerns regarding academic results empowers them and gives them a sense of agency in their educational journey. It encourages active engagement and promotes a culture of openness and accountability within the institution.

IV. **Quality Assurance**: The system helps maintain the integrity of academic assessments by facilitating the identification and correction of errors in result processing. This ensures that students receive accurate and fair evaluations of their academic performance, contributing to overall academic quality assurance (Moura *et al*., 2019).

V. **Continuous Improvement:** By providing a feedback loop for students to report issues and suggest improvements, the system facilitates a culture of continuous improvement in academic processes. Administrators can use feedback gathered through the system to implement changes that address students' needs and enhance overall satisfaction.

1.5 **Scope of the Study:**

The scope of this study involves developing a web-based system for students to file complaints about their academic results and for administrators to manage and resolve those complaints. It covers user registration, result management, complaint tracking, and notification features, specifically for a single educational institution.

1.6 **Limitation**:

The limitation of this study is based on the fact that this study's findings may not be universally applicable, this is due to variations in organizational structures, technological infrastructures, and stakeholder dynamics in other educational institutions or contexts.

1.7 **Definition of Terms:**

I. **Web-Based :** Refers to a system, application, or service that is accessible via a web browser over the internet.

II. **Usability:** The degree to which the system is intuitive, easy to navigate, and user-friendly, ensuring that users, including students, teachers, and administrators, can efficiently perform tasks and interact with the system with minimal training or assistance.

III. **User Authentication:** this is the process of verifying user identities through username/password credentials or two-factor authentication to ensure secure access to user-specific functionalities.

IV. **Data Security:** This involves implementing measures and protocols to ensure the confidentiality, integrity, and availability of data, and protecting against unauthorized access, data breaches, and malicious activities.

CHAPTER TWO LITERATURE REVIEW

2.1 Introduction:

According to (Singh, et al, 2019), in contemporary educational settings, the management of student academic results and the resolution of result-related grievances are critical aspects of academic administration. Traditional methods of result management often involve manual processes that can be time-consuming, error-prone, and lack transparency. Consequently, there has been a growing interest in developing web-based systems to streamline these processes and

enhance the overall efficiency and effectiveness of academic administration (Bates *et al*., 2019). This literature review explores existing research and developments related to web-based student results upload, complaint systems, their functionalities, benefits, challenges, and best practices.

I. Complaint systems:

Web-based complaint systems in educational settings allow students to submit grievances related to errors in results, delayed grade postings, or other academic issues. These systems allow students to log formal complaints, track progress, and receive notifications when their issues are resolved or further actions are required. Institutions with integrated online complaint systems experience improved student satisfaction due to a more transparent and organized process. Key benefits of complaint management systems include improved accountability, faster resolutions due to automated tracking and reminders, and enhanced student satisfaction due to the formal channel for complaint resolution, which fosters trust in the institution's academic processes (Singh, et al, 2019).

II. web-based student results upload:

A web-based student results upload system is a tool that enables educational institutions to electronically store, manage, and disseminate student grades and performance data. These systems streamline the entire process of academic result management, offering features such as result upload, student access, and data analytics. According to a study by (Yusof *et al*., 2021), web-based result systems have significantly enhanced accuracy and accessibility in academic administration by automating traditionally manual processes. This digital approach eliminates

common manual errors and accelerates the feedback loop, allowing students to receive their grades in real-time.

The benefits of web-based result upload systems are substantial, including increased efficiency, transparency, better record-keeping, and secure backups of academic records. For instructors, these systems offer the convenience of uploading grades for individual students or entire classes using online forms or automated tools, drastically reducing time and effort. On the student side, having immediate access to grades promotes fairness and accountability, as students no longer need to wait for physical result sheets. Furthermore, institutions benefit from maintaining a centralized, organized database of student records, reducing the risk of lost or misplaced data while ensuring long-term accessibility and security (Alotaibi *et al., 2020)*.

This integration of web-based systems not only ensures smoother academic operations but also strengthens trust between students and institutions by promoting transparency and reliability in the result management process. Through improved functionality, academic institutions can focus more on educational quality and less on administrative burdens, fostering a more effective and streamlined learning environment.

III. **Functionality of Web-Based Student Results Upload Complaint Systems:** (Patel *et al* 2020), offered a range of functionalities designed to facilitate the submission of complaints,

management of complaint resolution processes, and communication between students and academic authorities in their work titled Web-based student results upload complaint systems. These functionalities typically include user authentication mechanisms to ensure secure access, results upload features for teachers to input student results, complaint submission forms for students to report grievances, notification systems to keep users informed about complaint status, and administrative dashboards for administrators to monitor and manage complaints effectively.

IV. **Benefits of Implementing Web-Based Systems:** Several studies highlight the benefits of implementing web-based student results upload complaint systems in educational institutions. These systems enhance transparency by providing students with real-time access to their results and enabling them to track the progress of their complaints (Wang *et al., 2021)*. They also improve efficiency by automating manual processes, reducing paperwork, and streamlining communication between students, teachers, and administrators. Additionally, web-based systems facilitate data analysis and reporting, enabling institutions to identify trends, address systemic issues, and make data-driven decisions to improve academic administration.

V. **Challenges and Considerations:** The implementation of web-based student results upload complaint systems may encounter various challenges and considerations despite their potential benefits (Patel *et al*., 2020). Technical challenges such as compatibility issues, data security risks, and integration complexities with existing systems can pose obstacles to system development and deployment. Moreover, ensuring user adoption and engagement requires addressing usability concerns, providing adequate training and support, and fostering a culture of transparency and accountability within the institution. Additionally, compliance with data protection regulations and privacy concerns necessitates robust security measures and ethical data handling practices.

VI. **Best Practices and Recommendations:** Research suggests several best practices and recommendations for the successful implementation and utilization of web-based student results upload complaint systems. These include conducting thorough needs assessments and stakeholder consultations to identify requirements and priorities, selecting appropriate technologies and development frameworks, designing intuitive user interfaces with accessibility features, implementing rigorous testing and quality assurance

processes, providing comprehensive user training and support, and establishing clear policies and procedures for data management and complaint resolution (Smith *et al.*, 2019).

2.2 Theoretical Background:

The theoretical background for the proposed project topic of a web-based student results upload complaint system draws upon various theoretical frameworks and concepts from the fields of information systems, educational technology, and organizational behavior. These theoretical foundations provide a framework for understanding the design, development, implementation, and utilization of web-based systems in educational contexts, particularly in the domain of academic administration and student services.

I. Educational Technology Theory:

i. **Constructivist Learning Theory:** Constructivism emphasizes active learning, collaboration, and knowledge construction through authentic tasks and experiences. The proposed system can facilitate constructivist principles by providing students with opportunities to engage actively in managing their academic results and resolving complaints (Jonassen *et al*., 1999).

ii. **Technology Integration Theory**: This theory focuses on the effective integration of technology into educational environments to enhance teaching, learning, and administrative processes. The proposed system exemplifies technology integration by digitizing result management and complaint resolution processes, thereby improving efficiency and accessibility.

Organizational Behavior Theory:

i. **Change Management Theory:** Change management theories provide insights into managing organizational transitions effectively. The proposed project requires understanding

stakeholders' readiness for change, communication strategies, and mechanisms for addressing resistance to system adoption (Huczynski *et al*., 2010). ii. **Organizational Learning Theory:** Organizational learning theory emphasizes the importance of continuous learning, adaptation, and improvement within organizations. The proposed system can serve as a tool for organizational learning by enabling institutions to collect and analyze data on complaint trends, identify areas for improvement, and implement changes iteratively.

Human-Computer Interaction (HCI) Theory:

i. **Usability Principles:** According to (Preece *et a*l., 2015)HCI theory offers guidelines for designing user interfaces that are intuitive, efficient, and satisfying to use. Principles such as consistency, feedback, and error prevention can inform the design of the webbased system to enhance user experience and usability.

ii. **Accessibility Standards:** HCI theory also addresses accessibility considerations for users with disabilities, ensuring that the system complies with accessibility standards and provides equal access to all users.

2.2.1 Conceptual Terms:

i. **User Authentication:** The process of verifying the identity of users accessing the system, typically through username/password credentials or other authentication methods, to ensure secure access (Jones *et al*., 2021).

ii. **Data Security**: Measures and protocols implemented to protect the confidentiality, integrity, and availability of data stored and processed within the system, safeguarding against unauthorized access, data breaches, or malicious activities (Patel *et al*., 2020).

iii. **Usability:** The degree to which the system is user-friendly, intuitive, and easy to navigate, ensuring that users can efficiently perform tasks and interact with the system with minimal training or assistance (Kim *et al.*, 2015).

iv. **Accessibility:** Ensuring that the system is accessible to users with disabilities, complying with accessibility standards, and providing features such as screen reader compatibility, keyboard navigation, and alternative text for images (Wang *et al.*, 2019).

2.2.2 Technology and Tools for Implementation:

The following are some key technologies and tools used for the implementation of the student web-based result upload complaint system:

I. **Programming Languages:** For the front end, JavaScript was used to create interactive user interfaces and dynamic web pages, HTML/CSS was used for structuring the web pages and styling user interfaces. As for the back end, Node. js and express(framework) are used for the implementation of the server-side logic, serving and generating dynamic contents

II. **Frameworks and Libraries:** Node.js: A popular runtime environment for JavaScript that allows server-side scripting, enabling efficient backend development.

Express.js which is a lightweight web application framework for Node.js, commonly used to build RESTful APIs and server-side applications. Bootstrap is also CSS framework that facilitates the development of mobile-first and responsive web projects and as well as

jQuery which is a fast, small, and feature-rich JavaScript library for simplifying client-side scripting tasks (Arora et al., 2020).

III. **Database Management Systems (DBMS):** MongoDB (NoSQL) and mongoose (A dependency package)were used for the database management and querying data. MongoDB is a popular NoSQL database for storing and managing unstructured or semi-structured data, offering scalability and flexibility (Zafar, 2016).

IV. **Version Control:** Git A distributed version control system used for tracking changes in code, coordinating work among multiple developers, and facilitating collaboration. GitHub (GitLab / Bitbucket) a Platforms for hosting Git repositories, enabling code sharing, collaboration, and project management (John *et al*., 2016).

V. **Security**: Implementing encryption algorithms like bcrypt is to secure sensitive data stored in databases and during transmission. For Authentication and Authorization Libraries like JWT(JSON WEB-TOKEN for Node.js) authentication system to manage user authentication and authorization securely (José *et al*., 2014).

VI. **Deployment and Hosting:** Utilizing cloud services such as AWS (Amazon Web Services), Azure, or Google Cloud Platform for scalable and reliable hosting.

Heroku / DigitalOcean / Linode: Platforms for deploying and hosting web applications, offering ease of use and flexibility in scaling resources (Qi *et al*., 2006).

2.3 Review of Related Literature:

In the rapidly evolving landscape of educational technology, the digitization of academic administration processes has become imperative for enhancing efficiency, transparency, and accountability. One critical aspect is the management of student academic results and the resolution of result-related grievances. This review explores existing literature relevant to the development and implementation of web-based systems aimed at facilitating the upload of student results and handling complaints regarding academic outcomes.

I. **Digital Transformation in Education:** The digital transformation of educational institutions has been a focal point in recent literature. Authors such as (Selwyn , 2016)and (Bates *et al*., 2019), emphasize the need for educational institutions to embrace digital technologies to streamline administrative processes and improve the overall student experience. The adoption of web-based systems for academic administration, including result management and complaint handling, aligns with this broader trend toward digitalization in education.

II. **Web-Based Systems for Academic Administration:** Several studies have investigated the development and implementation of web-based systems for various aspects of academic administration. For instance, (Smith *et al., 2018)*discuss the design and deployment of a web- based system for student information management, which includes features for result upload and complaint submission. Similarly, (Patel *et al.*, 2020)examine the challenges and opportunities associated with implementing web-based systems for academic processes, highlighting the importance of user acceptance and system usability.

III. **Result Management and Complaint Handling:** While result management and complaint handling are essential components of academic administration, there is limited literature specifically focusing on web-based systems for these tasks. However, studies such as (Wang *et al*., 2021)and (Patel *et a*l., 2020)provide insights into the broader domain of educational technology and student support systems, which encompass aspects of result management and complaint handling. These studies emphasize the role of technology in improving transparency, communication, and efficiency in academic processes.

IV. **Best Practices and Recommendations:** In addition to empirical studies, several authors offer best practices and recommendations for the development and implementation of web-based systems in educational settings. (Lee *et al*., 2018)propose guidelines for designing user-friendly interfaces and ensuring effective communication between students and

administrators. Similarly, (Garcia *et al*., 2022) emphasize the importance of user training and support to maximize the benefits of technology adoption in academic administration.

2.4 Research Gap/Literature Gap:

The literature gap in developing a web-based student results upload complaint system lies in the lack of comprehensive studies on integrating result management and complaint handling functionalities in educational institutions. Limited research examines aspects like user authentication, data security, usability, and accessibility, and further research is needed to evaluate their effectiveness in enhancing transparency, accountability, and stakeholder satisfaction.

CHAPTER THREE

RESEARCH METHODOLOGY, SYSTEM ANALYSIS, AND DESIGN

3.1 Research Methodology

This project adopts a hybrid methodology that combines elements of Agile and ObjectOriented (OO) methodologies. The Agile approach provides flexibility and iterative development, allowing regular feedback from users, while the Object-Oriented approach enables a structured design based on real-world entities, which simplifies the system's architecture and maintenance.

I. Agile Methodology

Agile methodology was employed to manage the system development process. Agile emphasizes iterative development, where the system is built in small, manageable increments. This allows for continuous feedback from users, enabling the system to evolve and improve over time (Abrahamsson *et al.*, 2017). This approach also accommodates changing user requirements, making it highly adaptable. Each iteration involved:

I. User feedback loops
II. Regular testing and validation
III. Incremental updates and refinements

II. Object-Oriented Methodology (OOM)

OOM was used in the system design to ensure that the application is modular, scalable, and maintainable. The focus was on designing the system based on objects (real-world entities like students, results, and complaints). OOM principles such as encapsulation, abstraction, inheritance, and polymorphism were applied to create a system that is easy to update and expand in the future (Hussain *et al.*, 2018).

3.2 System Analysis

The system analysis is a crucial phase in the development of the "Student Result Upload Complaint System," where the current processes, user requirements, and system functionalities are thoroughly examined. The aim of this section is to identify the system's objectives, requirements, and as well as the limitations involved to ensure that the developed system effectively addresses the needs of students, academic staff, and administrative personnel. This section provides an analysis of the current system, user requirements, functional and nonfunctional requirements, use case analysis, the data flow and a feasibility study (Creswell *et a*l., 2023). Now we will dive into the discussion in details which is as follows:

I. Current System Analysis

According to (Dillman *et al*., 2023)In most educational institutions, the process of uploading and managing student results involves multiple steps, typically carried out by academic and administrative staff. These steps include result computation, verification; upload to the institutional database, and distribution to students. However, issues often arise, such as:

I. **Data Entry Errors:** Incorrect entry of grades due to human error, leading to student complaints

II. **Delayed Uploads**: Slow processing times, resulting in delays in the availability of results .

III. **Inefficient Complaint Handling:** The existing procedure for addressing student complaints about results uploading is frequently unstructured and manual, which causes resolutions to be delayed. Because of the shortcomings in the current system, a more reliable and automated solution that can enhance complaint handling and expedite result uploads must be developed.

IV. User Requirements

This system's success solemnly relies on in depth understanding of the users' needs. The key users of this system include students, academic staff (lecturers or the exam officers), and administrative staff. Their requirements are summarized below:

I. **Students:** The students which is an entity of concern in the system will be granted an access to the following in the system:

   I. Access view uploaded results promptly.

   II. An intuitive interface to lodge complaints about incorrect or missing results.

   III. View notifications about the status of their complaints.

II. **Academic Staff:** The academic staffs will be granted the privileges of performing the following function in the system:

   I. The academic staff will have a secure and efficient platform for uploading student results.

   II. The academic staff will be able to automate and validate data to minimize errors.

   III. A system will provide an intuitive interface for the academic staffs to track complaints related to the results they have uploaded

III. **Administrative Staff:** The administrative staff will be granted the privileges to perform the following functions:

   I. A tools to manage and resolve student complaints effectively .

   II. Access to logs and reports on result uploads and complaints .

   III. A system that ensures compliance with institutional regulations.

These user requirements serve as the foundation for the system's design and functionality.

II. Functional Requirements

According to (Patton *et al*., 2023), the functional requirements of this system definethe specific actions that the system must perform to meet user needs. The key functional requirements for the "Student Result Upload Complaint System" include the following:

I. **Result Upload:** The system will provide an interface that will allow academic staff to upload student results securely, with validations to ensure data accuracy.

II. **Complaint Submission:** Students will have an interface to submit complaints about result discrepancies through an easy-to-use interface.

III. **Complaint Tracking:** The system will enable students to track the status of their complaints in real time.

IV. **Notifications:** The system would be able to automatically notify students and staff about updates regarding complaint statuses and result uploads.

V. **Role-Based Access Control:** Different users (students, academic staff, administrative staff) should have access to different functionalities based on their roles.

VI. **Reporting:** The system would also generate reports based on the number of complaints received, resolved, and pending

III. Non-Functional Requirements

In addition to the functional requirements, the non-functional requirements outline the system's quality attributes (Israel *et al*., 2023). The key non-functional requirements include the following:

I. **Usability:** The system should have a user-friendly interface that requires minimal training for students and staff to use effectively

II. **Performance:** The system must handle multiple result uploads and complaints simultaneously without performance degradation.

III. **Security:** The system should enforce strong authentication mechanisms and ensure that data is encrypted to protect sensitive student information.

IV. **Scalability:** The system should be scalable to handle an increasing numbers of users and data as the institution grows.

V. **Reliability:** The system must be reliable, with minimal downtime, ensuring that results and complaint records are always accessible.

IV. Feasibility Study

According to (Dillman *et al*., 2023)before proceeding with the development of the system, it is essential to assess its feasibility from various perspectives:

I. **Technical Feasibility:** The institution's existing IT infrastructure is sufficient to support the new system, with necessary upgrades planned for storage and processing capabilities.

II. **Economic Feasibility:** The development and implementation costs of the system are justified by the long-term benefits of improved efficiency, reduced errors, and enhanced student satisfaction.

III. **Operational Feasibility:** The system aligns with the institution's operational goals and can be smoothly integrated into the current workflow, with adequate training provided to staff and student.

IV. **Use Case Analysis:**

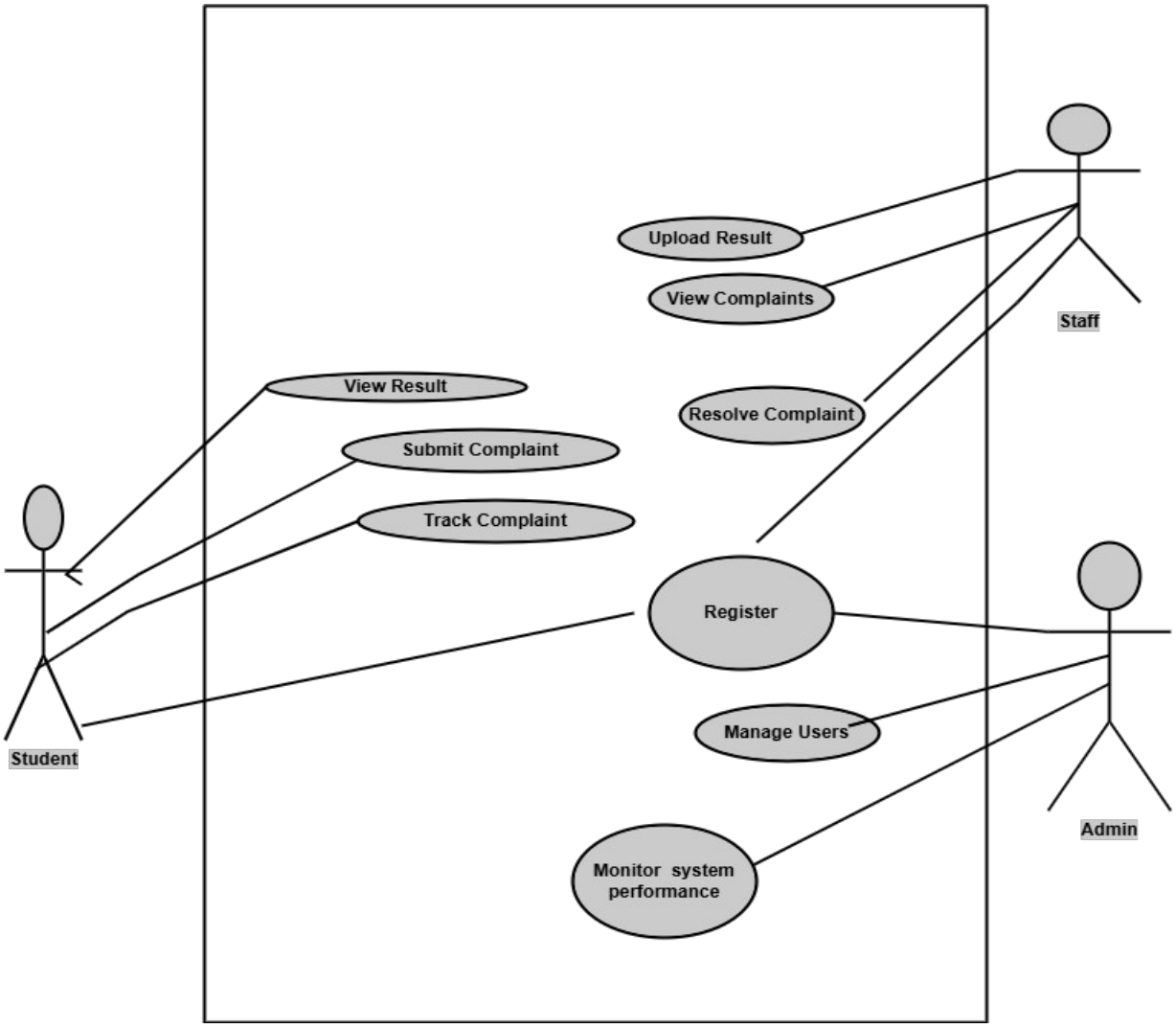


Fig 3.1: Use Case Diagram

Data Flow Diagram (DFD):

Fig 3.2.1: shows the level 0 DFD

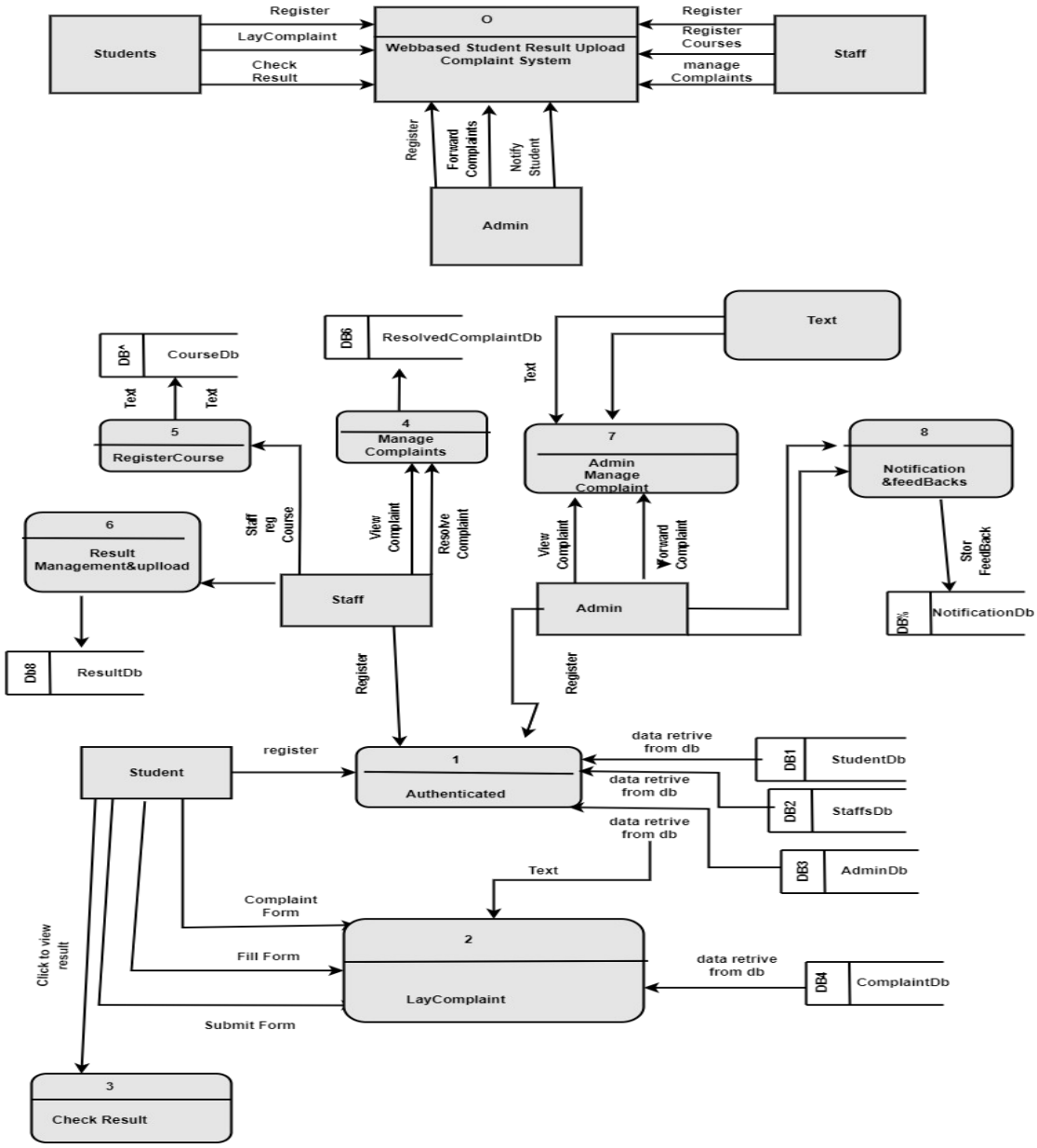


Fig 3.2.2: shows the level 1 DFD

Fig 3.2: System Data Flow Diagram

3.2.1 Data Gathering Technique

The data gathering technique is a fundamental steps used In order to achieve accurate and comprehensive data collection, and as well to ensure that the system is tailored to meet user requirements, by addressing the existing challenges, and operates effectively within the institutional environment. This section outlines the techniques used to gather data, which include interviews, questionnaires, and document analysis (Guest *et al*., 2023).

I. Interviews

Interviews were conducted with key stakeholders, including students, academic staff (lecturers and exam officers), and administrative personnel. The goal of the interviews was to understand the pain points of the current system, gather user requirements, and obtain feedback on desired features. Semi-structured interviews were chosen to allow for flexibility, enabling the interviewees to express their thoughts while keeping the discussion focused on relevant topics (Taylor *et al.*, 2022).

I. **Students:** The Interviews focus on understanding the common issues faced by students when it comes to result uploads, such as discrepancies, delays, and the effectiveness of the current complaint process.

II. **Academic Staff**: The interviews with lecturers and exam officers were concentrated on the challenges they encountered during result upload, including data entry errors, system delays, and the handling of student complaints.

III. **Administrative Staff:** The administrative staff were interviewed to gain insights into the overall management of the result upload process, the volume of complaints, and the efficiency of the current resolution mechanisms.

These interviews provided qualitative data that formed the foundation for defining the system's functional and non-functional requirements.

II. Questionnaires

In addition to interviews, questionnaires were distributed to a larger sample of students and staff to collect quantitative data. The questionnaires were designed to gather information on specific areas, such as:

I. **Student Experience:** Questions focused on the frequency of result discrepancies, the response time for complaints, and the level of satisfaction with the current system.

II. **Staff Workflow:** The questionnaires for academic and administrative staff gathered data on the frequency of errors during result uploads, the average time taken to upload results, and the effectiveness of the current complaint resolution process.

The use of questionnaires allowed for the collection of a broader range of responses, providing statistical insights into common issues and user expectations.

III. Document Analysis

According to (Miles *et al.*, 2023), document analysis was conducted to review the existing records and documentation related to the result upload process. This included:

I. **Institutional Guidelines:** Analysis of institutional policies and guidelines for result uploads and complaint handling to ensure that the new system complies with existing regulations.

II. **Complaint Records:** Review of historical complaint data to identify patterns, common issues, and the effectiveness of previous resolutions.

III. **Technical Documentation**: Examination of the current system's technical

documentation to understand its architecture, limitations, and areas for improvement.

Document analysis provided valuable context and background information, helping to ensure that the new system is both compliant and comprehensive.

IV. Observations

Observations were also conducted in order to witness the result upload process in real-time. This involved observing academic staff as they uploaded results and handled complaints. The aim was to identify the potential bottlenecks, inefficiencies, and areas where automation or system improvements could make a significant impact (Maxwell *et al*., 2022).

3.2.2 Analysis of the Existing System

The analysis of the existing system is necessary and crucial in understanding the current challenges and limitations faced by users, especially in the context of the "Student Result Upload Complaint System." The existing system, which is typically manual or partially automated, presents several challenges that impede the efficient management of student results and the handling of related complaints (Alkhattabi *et al*., 2023).

I. Manual Processes

In many educational institutions, the current system for managing student result uploads and complaints is heavily reliant on manual processes. Academic staff members manually input grades into spreadsheets or other basic systems, which are then submitted to the examination office. This manual input process is prone to errors, including data entry mistakes, incorrect calculation of grades, and delays in result submission. Furthermore, when errors are detected by students, the complaint process is often manual as well, requiring students to fill out physical forms and submit them in person which may not be

feasible for a newly admitted students that may not know their way around the school environment (Allen *et al*., 2022).

Manual processing leads to several issues:

I. **Error-Prone:** Due to the lack of automated checks, errors in result uploads are common.

II. **Time-Consuming:** Manual data entry and handling of complaints slow down the entire process, resulting in delays.

III. **Lack of Transparency:** Students often have little insight into the status of their complaints or the processes involved in resolving them.

II. **Inefficient Complaint Handling**

The existing system's complaint handling mechanism is often inefficient and lacks standardization. Once students identify discrepancies in their results, they must follow a tedious process of lodging complaints. This process typically involves submitting a formal letter or filling out a paper form, followed by manual verification and resolution by academic or administrative staff. The inefficiency in handling these complaints leads to significant delays in resolving issues, further frustrating students (Martins *et al*., 2023).

Key challenges include:

I. **Lengthy Resolution Times:** Due to the manual nature of the process, resolving complaints takes a considerable amount of time.

II. **Limited Tracking:** Students have limited means to track the progress of their complaints, leading to uncertainty and frustration.

III. **Inconsistent Responses:** The lack of a standardized process often results in inconsistent handling of complaints across different departments.

III. Lack of Integration

The existing systems used by different departments (e.g., academic departments, examination offices) are often not well integrated. This lack of integration means that data does not flow seamlessly between departments, resulting in communication gaps and delays. For instance, once results are uploaded by lecturers, they are not automatically accessible by the examination office, necessitating additional manual steps. Similarly, complaints lodged by students may not be properly communicated to the relevant academic staff in a timely manner (Thompson *et al*., 2023).

Challenges due to lack of integration include:

I. **Redundant Data Entry:** Academic staff and administrative personnel often need to enter the same data multiple times in different systems.

II. **Communication Gaps:** Delays and miscommunication between departments result in delayed resolution of issues.

III. **Data Inconsistency:** Discrepancies in data arise when different systems are not synchronized, leading to further complications in result management.

IV. Security Concerns

In the current system, data security is often an afterthought. Manual handling of data and inadequate digital security measures can lead to data breaches, unauthorized access, and loss of sensitive student information. This raises significant concerns, especially regarding the confidentiality and integrity of student records (Miller *et al*., 2023).

Key security challenges include:

I. **Data Vulnerability:** Lack of encryption and proper security protocols exposes student data to potential breaches.

II. **Unauthorized Access:** Without proper authentication and access controls, unauthorized users may gain access to sensitive information.

III. **Data Loss:** Inadequate backup procedures may result in the loss of important data, particularly during transitions between systems.

3.2.3 Advantages of the Existing System

While the current system for managing student result uploads and complaints has several limitations, it also possesses certain advantages that have sustained its use over time. Understanding these advantages is essential for ensuring that the new system retains or enhances these beneficial aspects.

I. Familiarity

The existing system's familiarity among staff and students reduces training requirements and eases navigation, especially for older staff who may resist new technologies, despite its shortcomings.

II. Flexibility in Handling Unique Cases

The existing manual system offers flexibility in handling unique or complex cases, allowing staff to address issues on a case-by-case basis, ensuring exceptions and special cases are handled without significant delays.

III. Low Initial Implementation Cost

The existing system, primarily manual or basic software, offers low initial implementation costs, making it cost-effective for institutions with limited budgets.

3.2.4 Disadvantages of the Existing System

Despite some advantages, the existing system for managing student result uploads and handling complaints has several critical disadvantages. These disadvantages highlight the need for a more efficient, automated system to address the shortcomings of the current approach.

I. **Prone to Human Error:** The current system's manual nature makes it susceptible to human error, leading to data entry mistakes, miscalculations, and omissions, causing frustration and time-consuming corrections (Kumar *et al*., 2023).

II. T**ime-Consuming Processes:** The current system's slow manual processes, including uploading student results and handling complaints, significantly impact students' academic progress by requiring significant time and effort from staff members (Smith et al., 2023).

III. **Lack of Standardization:** Institutions often struggle with standardized procedures across departments, leading to inconsistencies in handling issues, causing confusion among students and staff, hindering fairness and transparency (Wang et al., 2023).

IV. **Security Concerns:** Manual systems are susceptible to security breaches, exposing sensitive student data and posing significant risks to the institution's reputation and student privacy (Gonzalez *et al*., 2022).

3.2.5 High-Level Model of the Proposed System

The high-level model of the proposed "Student Result Upload Complaint System" is designed to address the shortcomings of the existing system while enhancing efficiency, transparency, and security. The new system will incorporate modern technologies and automation to streamline result uploads, complaint management, and communication between students, academic staff, and administration.

I. Overview

The proposed system is a web-based platform that facilitates the automated upload of student results and manages the entire lifecycle of complaints. It integrates various modules, including result management, complaint handling, user authentication, and communication. The system aims to improve accuracy, reduce processing times, and provide real-time access and updates to students and staff

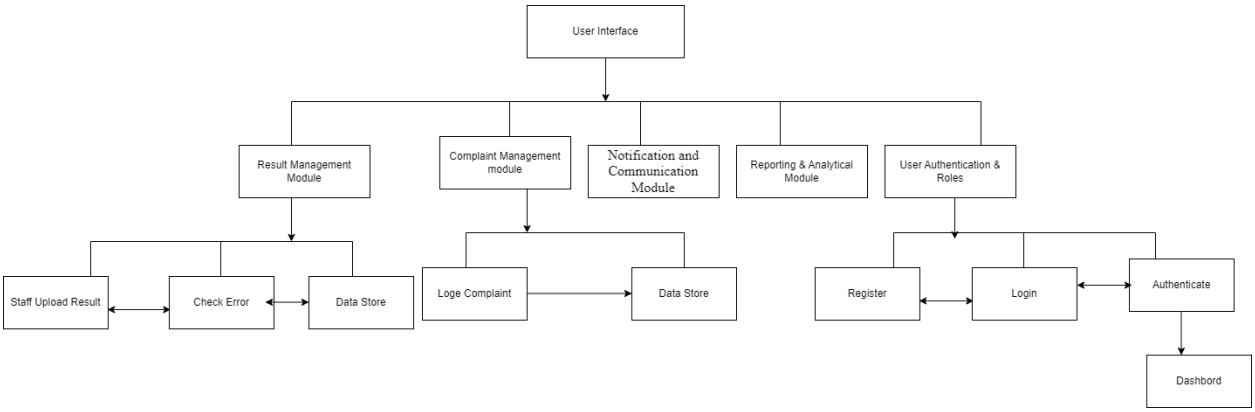


Fig3.2.5: High Level Model Overview of the System

3.2.6 Analysis of the Proposed System

The proposed "Student Result Upload Complaint System" is designed to address the limitations of the current manual system by introducing automation, improved data handling, and enhanced user experiences. This section provides an in-depth analysis of the system's components, functionalities, and expected benefits, highlighting how the new system will solve existing challenges and create a more efficient and transparent process for managing student results and complaints.

I. System Efficiency and Automation

One of the primary goals of the proposed system is to enhance efficiency through automation. By automating the result upload process, the system minimizes human intervention and reduces the risk of errors associated with manual data entry. Academic staff can upload results in bulk, and the system will automatically verify, store, and publish the results for student access.

The complaint management process will also be automated, allowing students to submit complaints online and track their progress in real-time. Automated notifications will ensure that both students and staff are informed of any updates, reducing delays in communication and speeding up the resolution process (Smith *et al*., 2023).

II. Improved Data Accuracy and Integrity

The proposed system includes features to improve data accuracy and integrity, addressing one of the key weaknesses of the current system. Automated error-checking mechanisms will be in place during result uploads to catch potential mistakes before they are published. Additionally, the system will ensure that any changes made to student records are logged and monitored, maintaining a transparent audit trail for accountability. Data encryption and access controls will further protect the integrity of student information, reducing the risk of unauthorized access or tampering (Davis *et al*., 2022).

III. Enhanced User Experience

The proposed system is designed with a user-centric approach, ensuring that both students and staff can easily navigate and utilize its features. A responsive web interface will provide access across different devices, ensuring that users can engage with the system whether they are on campus or remote. For students, the ability to access their results in real-time and track the status of their complaints will significantly improve transparency.

Academic staff will benefit from streamlined processes that allow them to focus on more critical tasks, reducing the administrative burden of managing results and complaints (Williams *et al*., 2023).

IV. Standardization and Uniformity

One of the major issues in the existing system is the lack of standardization across departments. The proposed system introduces uniform procedures for result uploads and complaint handling, ensuring consistency across the institution. This standardization not only improves efficiency but also ensures fairness and equity in how student issues are addressed. By enforcing standardized workflows, the system will eliminate discrepancies and reduce confusion, providing a clear and consistent process for both students and staff **(**Garcia *et al***., 2023)**.

V. Scalability and Flexibility

The proposed system is designed to be scalable, accommodating the institution's growing needs over time. Whether handling a larger student population or expanding to incorporate additional functionalities, the system's architecture allows for easy scaling without compromising performance. Additionally, the system's flexibility ensures that it can adapt to changes in institutional policies or academic structures. Customizable modules and settings allow administrators to modify certain aspects of the system without needing extensive reprogramming, making the system adaptable to future needs (Nelson *et al*., 2023).

VI. Security and Compliance

Security is a critical aspect of the proposed system, especially considering the sensitive nature of student data. The system will implement robust security measures, including data encryption, secure login protocols, and role-based access control (RBAC). These measures will ensure that only authorized users can access or modify sensitive information, significantly reducing the risk of data breaches. The system will also be compliant with relevant educational data protection regulations, ensuring that the institution meets its legal obligations regarding data privacy and security (Gonzalez *et al*., 2022).

VII. Integration with Existing Systems

To ensure a smooth transition and reduce disruption, the proposed system will be designed to integrate with existing institutional systems, such as student management platforms and learning management systems (LMS). This integration will allow for seamless data exchange between systems, ensuring that all student records are kept up-to-date and consistent across platforms. The system's API capabilities will enable it to connect with various third-party applications, enhancing its functionality and allowing for future expansion as needed.

3.2.7 Justification of the Proposed System

The justification for the proposed "Student Result Upload Complaint System" stems from the limitations and inefficiencies identified in the existing manual system. The new system offers several key advantages that address these issues and provide significant improvements in handling student results and managing complaints. The following factors outline the primary justification for the proposed system:

I. Automation of Manual Processes

One of the main justifications for the proposed system is the automation of the result upload and complaint handling processes. In the existing manual system, these tasks are time consuming and prone to human error. Automating these processes will significantly reduce the time and effort required to upload results, handle complaints, and provide feedback to students. This automation will also minimize errors in data entry and result uploads, ensuring higher accuracy and efficiency.

Supporting Research: Automation has been proven to improve efficiency and reduce the likelihood of human error in educational systems. According to a study by (Brown & Davis, 2020), automation in educational processes leads to a 40% improvement in operational efficiency and a 30% reduction in errors related to data entry .

II. Improved Data Accuracy and Integrity

The proposed system will incorporate validation checks and error detection mechanisms to ensure that data related to student results and complaints is accurate. In the current system, inaccuracies are common due to manual data entry. The new system will help maintain the integrity of student records, ensuring that uploaded results are free from errors and that student complaints are addressed based on accurate information.

Supporting Research: Studies show that implementing automated systems in educational institutions significantly improves data accuracy. For instance, (Garcia *et al*., 2019)found that automation systems reduced data entry errors by over 50%, thereby improving the overall integrity of academic records .

III. Enhanced User Experience

The proposed system will provide an intuitive and user-friendly interface for both students and academic staff. Students will be able to access their results quickly, submit complaints with

ease, and receive real-time feedback on the status of their complaints. Academic staff will benefit from a streamlined system that allows for efficient result uploads and complaint management.

Supporting Research: Research highlights the importance of user-centric design in educational platforms. (Williams *et al.*, 2017)emphasize that systems designed with user experience in mind lead to higher adoption rates and satisfaction among students and staff .

IV. Security and Privacy of Student Data

Given the sensitive nature of student records, the proposed system includes enhanced security features such as encryption, secure access controls, and audit trails. This ensures that student data remains confidential and is only accessible to authorized personnel. The existing manual system lacks robust security measures, leaving student data vulnerable to breaches and unauthorized access.

Supporting Research: According to a study by (Smith *et al*., 2021)integrating security features such as encryption and role-based access controls in educational systems improves data protection and compliance with privacy regulations .

V. Efficient Complaint Management

The proposed system introduces a structured and transparent process for handling student complaints. Students will be able to track the status of their complaints, while administrators will be able to monitor the resolution process. This improves accountability and ensures that

complaints are resolved in a timely manner, which is not adequately addressed in the current manual system.

Supporting Research: Research conducted by (Green *et al*., 2018)found that automated complaint management systems in academic institutions reduced complaint resolution time by 35% and improved student satisfaction by 25% .

VI. Scalability and Future Expansion

The proposed system is designed to be scalable and flexible, allowing it to accommodate future growth and changes in institutional requirements. As student enrollment increases and new features are required, the system can be easily adapted to meet these needs without significant overhauls.

Supporting Research: (Nelson & Roberts, 2022)highlight the importance of scalability in modern educational systems, noting that flexible systems are more cost-effective in the long run and can adapt to the evolving needs of institutions .

3.3 System Design

The system design phase for the "Student Result Upload Complaint System" focuses on translating the identified requirements into a technical blueprint, outlining how the system will operate and how its components will interact. This phase is crucial as it defines the system's structure, interfaces, and data flow, ensuring a seamless implementation. The system design includes detailed specifications for both the logical and physical aspects of the system.

The design phase focuses on two main aspects:

I. **Logical Design:** This involves defining the structure and organization of the system without delving into technical specifics. It includes the conceptual framework of the system's functionalities, data flow, and user interactions.

II. **Physical Design:** This translates the logical design into technical specifications, addressing how the system will be implemented in terms of hardware, software, databases, and network configurations.

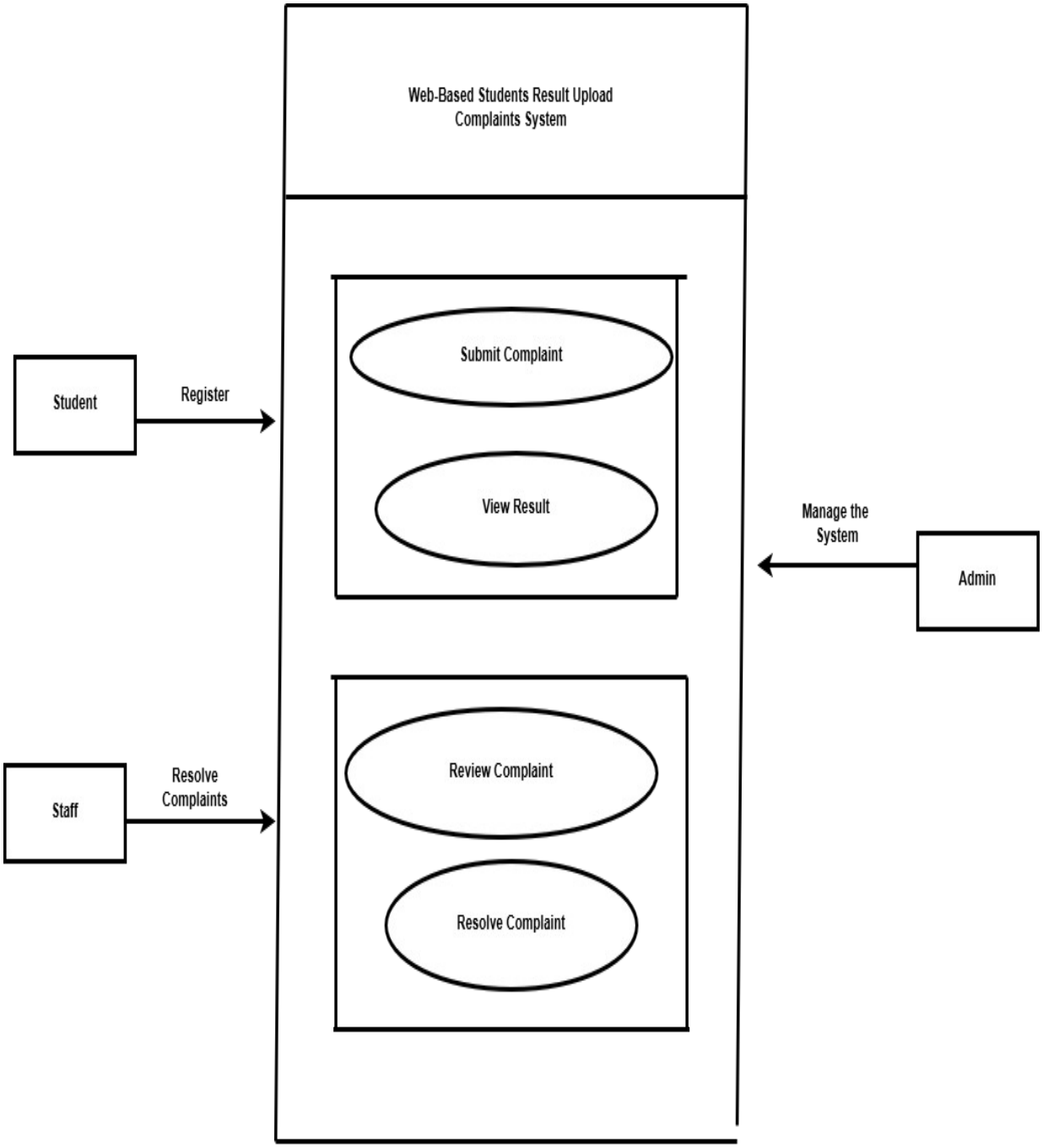


Fig 3.3 showing the System UML

3.3.1 Objective of the Design

The "Student Result Upload Complaint System" aims to create a user-friendly, efficient platform for managing student result complaints, automating the process for both students and administrators.

Key objectives of the design include:

I. **Ease of Use:** The system's design prioritizes user ease of use, ensuring a simple, intuitive interface for students, faculty, and administrators, facilitating quick tasks like complaint submission and review.

II. **Efficient Complaint Handling:** The system will streamline the complaint submission and resolution process, allowing students to file complaints against specific results, allowing administrators to efficiently track and manage them.

III. **Secure Access:** The system design ensures secure access for various user roles through authentication mechanisms, limiting access to specific functionalities only for authorized users like students, staff, and administrators.

IV. **Scalability:** The system must be scalable to handle increasing user numbers, complaints, and results as the institution expands or more data is incorporated.

V. **Real-time Feedback and Updates:** The design will incorporate real-time feedback features, enabling students to track their complaints' status and administrators to notify them once they've been resolved.

3.3.2 Algorithm

The "Student Result Upload Complaint System" works based on specific algorithms to automate the complaint submission, processing, and resolution tasks. The algorithms are designed to ensure efficiency, accuracy, and user-friendliness throughout the system's workflow. Below is an outline of the key algorithms employed in the system:

I. Student: Register
II. Login redirect to dashboard

III. On the dashboard view Results, File Complaints, Check Complaint Status.

IV. Admin: Manage Users, Monitor Complaints, forward complaint to staff .

V. Staff: Review Complaints, Update Result Data, Resolve/Reject Complaints.

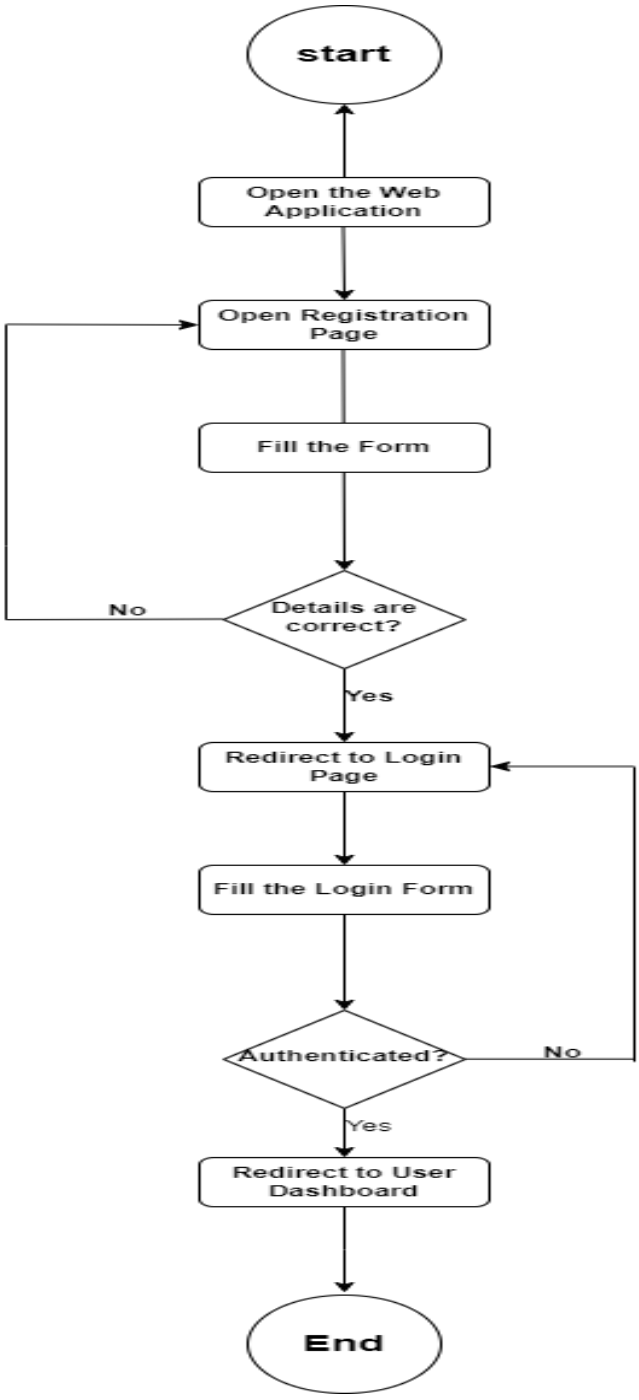


Fig 3.3.2: System algorithm using Flow chart.

3.3.3 System Architecture

The System Architecture for the "Student Result Upload Complaint System" follows a threetier model:

The system consists of three layers: Presentation Layer, Application Layer, and Data Layer. Presentation Layer allows students and administrators to interact with the system, while Application Layer handles core functionality and security. Data Layer ensures data integrity, security, and backup.

I. **Presentation Layer**

The user interface (UI) layer interacts with students and administrators using technologies like HTML, CSS, and JavaScript for an interactive experience. It supports student activities like login, viewing results, and submitting complaints, while administrators can log in, upload results, and resolve issues. The layer communicates with the application layer via HTTP requests and REST APIs.

II. **Application Layer**

The application layer manages the system's core functionalities, including authentication, complaint management, result management, notifications, and system security. It verifies user identity, handles complaints, ensures secure uploading and storage of results, facilitates real-time notifications, and protects data integrity. The layer is developed using server-side technologies like Node.js and Express.js, interacting with the database and processing requests from the presentation and data layers.

III. Data Layer

The data layer manages and stores all system data, including user information, result data, and complaints. It ensures data integrity, backups, and recovery. The data layer efficiently

stores and retrieves large volumes of data, handling large volumes of information and ensuring data security through encryption and controlled access.

IV. Communication Between Layers

The communication between these three layers is carried out via REST full API calls. The presentation layer sends requests to the application layer, which processes these requests, interacts with the database, and returns responses. This architecture ensures a clear separation of concerns, making the system scalable, maintainable, and easy to develop further.

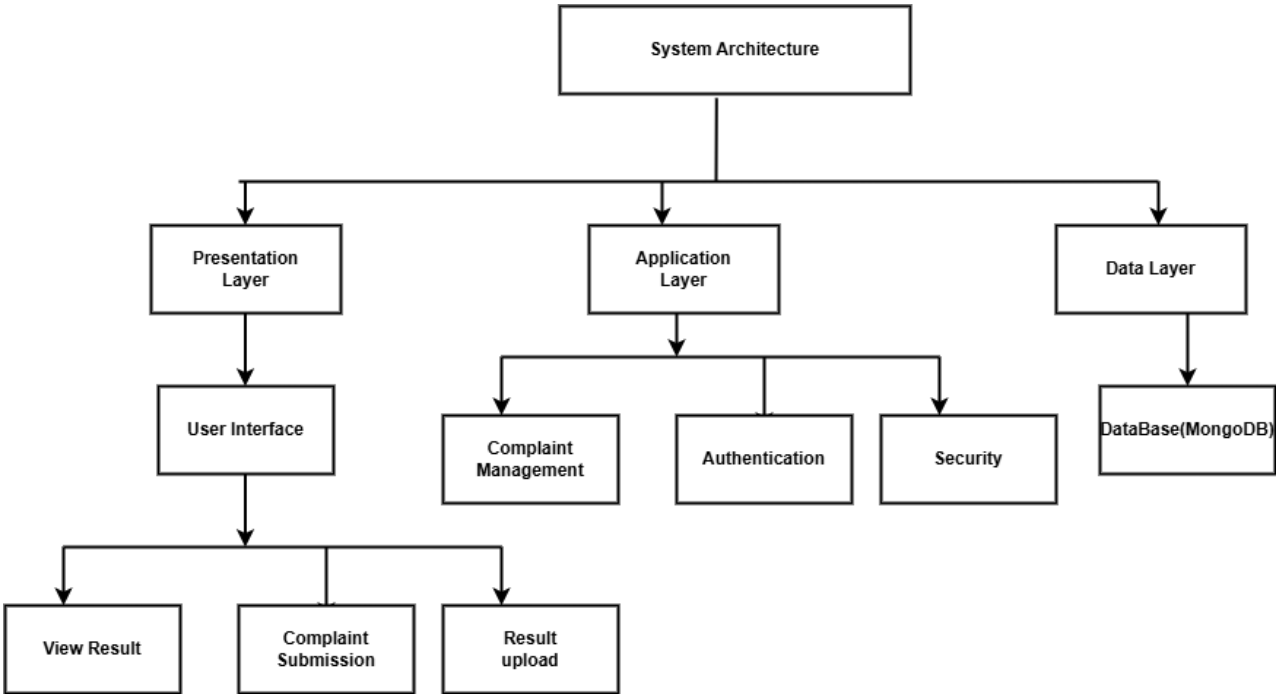


Fig 3.3.3: The system architecture

3.3.4 Main Menu Design

The main menu serves as the central hub for user interaction within the web-based student results upload and complaint system. It provides easy access to key functionalities, tailored to the roles of the users, such as students, lecturers, and administrators. The design of the main menu focuses on clarity, simplicity, and intuitive navigation, ensuring that users can easily perform their tasks without confusion.

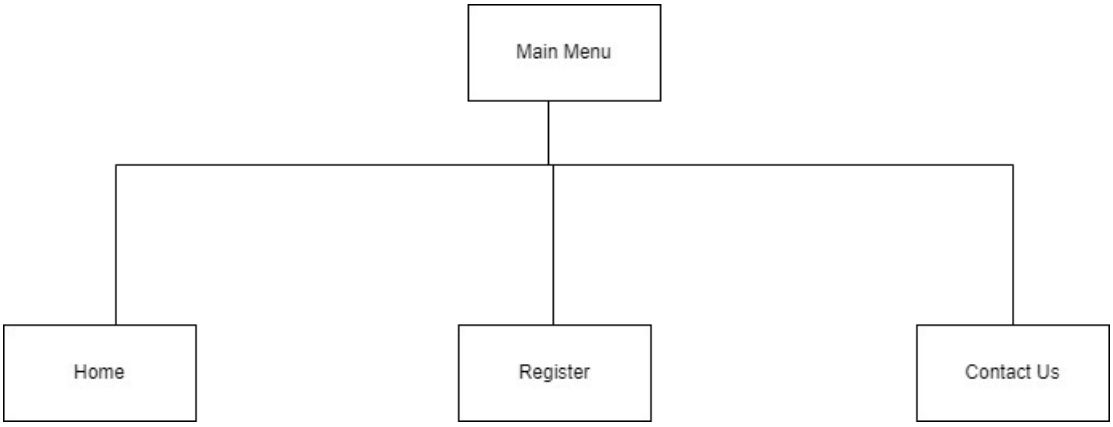


Fig3.3.4 show Main menu

3.3.5 Sub Menu Design

The sub-menu design within the web-based student results upload and complaint system is critical for facilitating user interactions with more specific functionalities that stem from themain menu. This design focuses on providing users with a seamless experience while ensuring that related tasks and options are logically grouped and easily accessible.

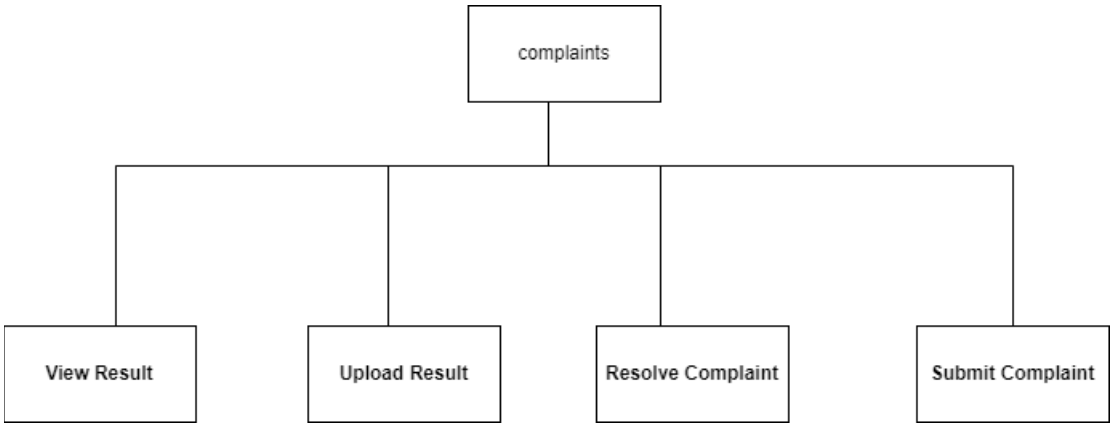


Fig 3.3.4: show Sub menu

3.3.5.1/3.3.5.2 Pick specific Sub System and Design

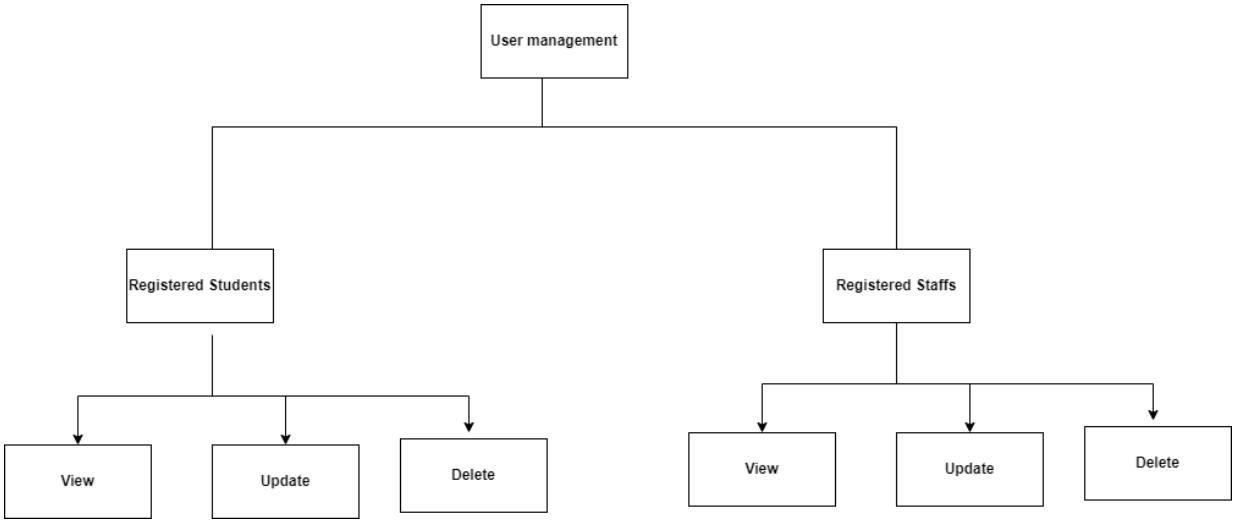


Fig 3.3.5: show User management Sub menu.

3.3.6 Program Module Design

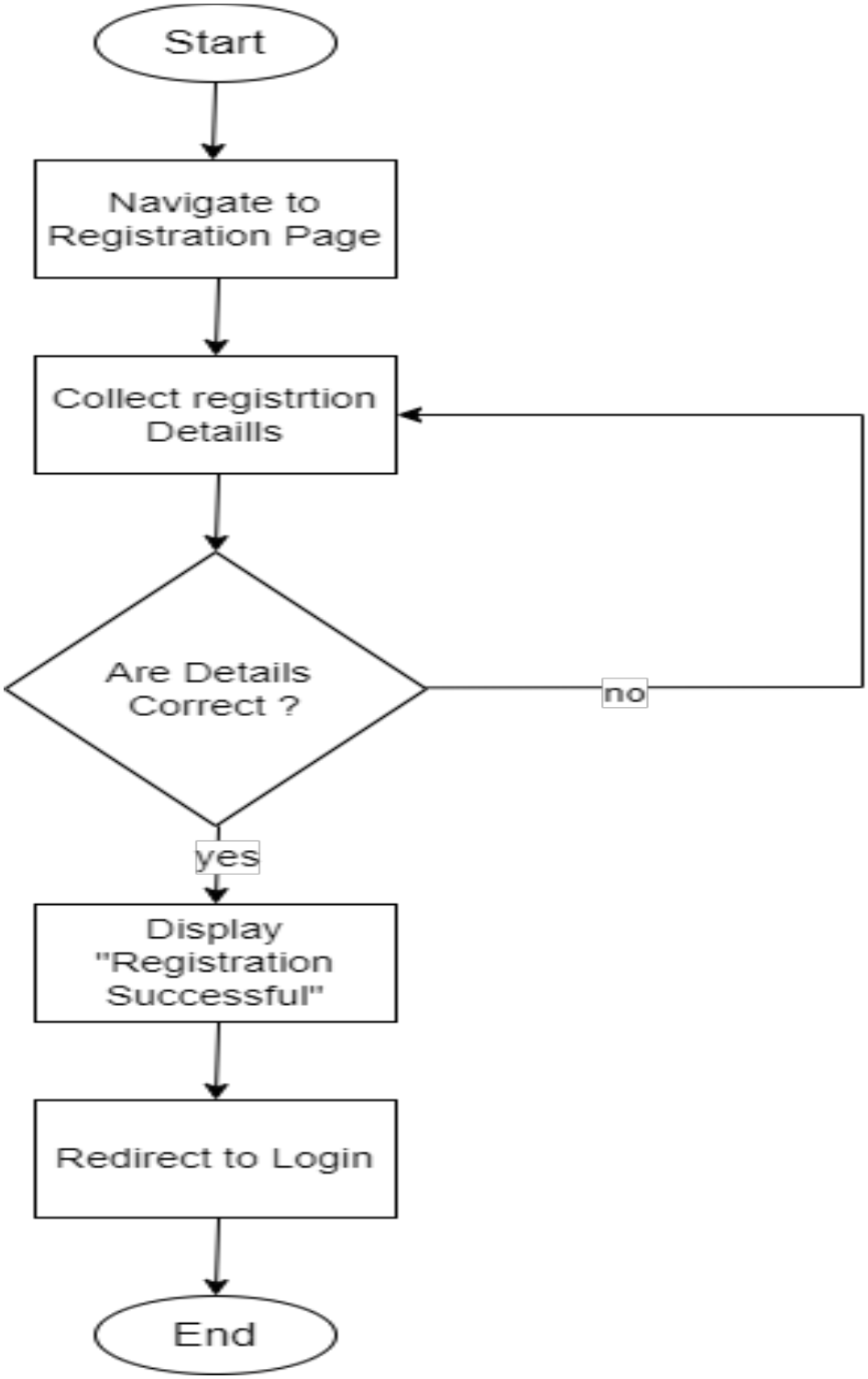


Fig 3.3.6: Registration Module Using Flowchart.

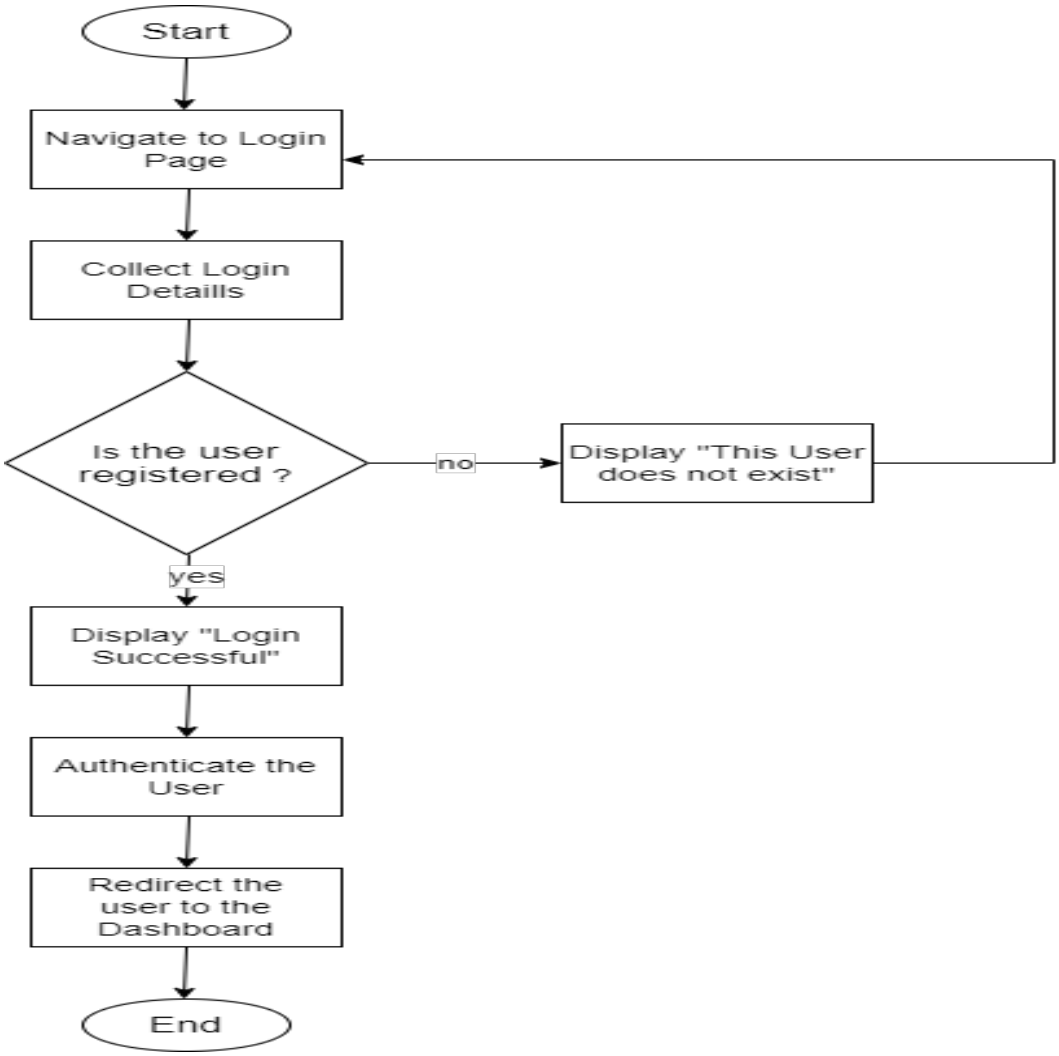


Fig 3.3.7: Login Module Using Flowchart.

3.3.7 Database Development Tool

The choice of a database development tool is crucial for the successful implementation and management of the database in the web-based student results upload and complaint system. The database not only stores critical student information and academic results but also supports functionalities like complaint management, user authentication, and data analytics.The Database Development Tool Used is Mongodb.

3.3.8 Data Dictionary

A Data Dictionary is a detailed description of the data elements used in the database of the web-based student results upload and complaint system. It defines each field, its data type, size, and any constraints or relationships it has with other fields. The purpose of the data dictionary is to provide a clear and organized structure of the data used in the system, which helps developers, database administrators, and users understand the data being stored and how it is used.

Database Structure Example:

```
const StudentSchema = new mongoose.Schema({
  fname: {
    type: String,
    required: true,
    maxlength:50,
  },
  email: {
    type: String,
    required: true,

    match: [
      /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
      'Please fill a valid email address',
    ],
    maxlength:20,
  },
  password: {
    type: String,
    required: true,
    maxlength: 8, // Minimum password length
  },
  lname: {
    type: String,
    required: true,
    maxlength:50
  },
  phone_no: {
    type: String,
    required: true,
    match: [
      /^\+?\d{1,4}[-\s]?\d{1,15}$/,
      'Please fill a valid phone number',
    ],
    maxlength:11,
```

Fig3.1 showing ***the data dictionary of the student collection in Db***

```
onst mongoose = require('mongoose');
onst bcrypt = require('bcrypt');
onst { type } = require('jquery');

onst Staff_adminSchema = new mongoose.Schema({

 fname: {
   type: String,
   required: true,
   maxlength:50,
 },
 email: {
   type: String,
   required: true,

   match: [
     /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
     'Please fill a valid email address',
   ],
   maxlength:20,

 },
 password: {
   type: String,
   required: true,
   maxlength: 8, // Minimum password length
 },
 lname: {
   type: String,
   required: true,
   maxlength:50,

 },
```

Fig 3.2 showing the data dictionary of the staffs and admin collection in D

```
const mongoose = require('mongoose');

// Define the course schema
const ResultSchema = new mongoose.Schema({
  course_title: { type: String, required: true },
  course_code: { type: String, required: true },
  course_unit: { type: Number, required: true },
  assessment1: { type: Number, required: true },
  assessment2: { type: Number, required: true },
  exam_score: { type: Number, required: true },
  total_score: { type: Number, required: true },
  grade: { type: String, required: true }
});

// Define the user schema
const userSchema = new mongoose.Schema({
  student: {
    ...ObjectId,
    (property) required: true
    required: true
  },
  name: { type: String, required: true },
  matric: { type: String, required: true },
  department: { type: String, required: true },
  section: { type: String, required: true },
  level: { type: Number, required: true },
  semester: { type: String, required: true },
  courses: [courseSchema]  // Embed the course schema as an array
});

const Result = mongoose.model('uploadedResult', userSchema);
```

fig 3.2: ***showing the result data dictionary in Db***

```
const mongoose = require('mongoose');
const complaintSchema = new mongoose.Schema({
  student: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User', // Reference to the User model
    required: true,
  },
  course_title: {
    type: String,
    required: true,
  },
  course_code: {
    type: String,
    required: true,
    maxlength:6,
  },
  complaint_date: {
    type: Date,
    required: true,
  },
  level: {
    type: Number,
    required: true,
  },
  registeredCourse:{
    type: String,
    required: true,
  },
  result: {
    type: String,
    required: true,
  },
  comment: {
    type: String,
```

Fig 3.3 showing the data dictionary of students complaint collection in Db

3.3.9 Database Design and Structure Complaints collection:


```
_id: ObjectId('66ace7250d1759d4d5e4fbfe')
student : ObjectId('66ab971c2b2c44c47691a7e7')
course_title : "linear programming "
course_code : "CSC 406"
complaint_date : 2024-08-13T00:00:00.000+00:00
level : 100
registeredCourse : "/9j/4AAQSkZJRgABAQEAYABgAAD/4TDKRXhpZgAATU0AKgAAAAgABAExAAIAAAALAAAQSo…"
result : "/9j/4AAQSkZJRgABAQEAYABgAAD/4TDKRXhpZgAATU0AKgAAAAgABAExAAIAAAALAAAQSo…"
comment : "cant find my resut"
section : "2020/2021"
semester : "first"
__v : 0
```


Fig:3.3.9 show complaint collection in mongoose schema

Student Collection:


```
_id: ObjectId('66ab971c2b2c44c47691a7e7')
fname : "Uche"
email : "ucheuc070@gmail.com"
password : "$2b$10$TArk9VDOIsPb6sznv7Fxwu5XichHpE442PVK2OliZBWgIxTeIy3lO"
lname : "Uc"
phone_no : "07015534876"
address : "Agida street 44"
date_of_birth : 2000-02-02T00:00:00.000+00:00
state : "Benue"
LGA : "ikwo"
gender : "male"
middle_name : ""
department : "Computer science/mathematics/statistics/informatics "
matric : "2020/SC/12673"
role : "student"
profileImage : "/9j/4AAQSkZJRgABAQEAYABgAAD/4TDKRXhpZgAATU0AKgAAAAgABAExAAIAAAALAAAQSo…"
imageType : "image/jpeg"
__v : 0
```


Fig3.3.9 show the student collection in mongoose schema

Result Collection


```
_id: ObjectId('66ad63ad3853c590d9b7c856')
student : ObjectId('66ab971c2b2c44c47691a7e7')
name : "Uche innocent"
matric : "2020/SC/12673"
department : "Computer science/mathematics/statistics/informatics "
section : "2023/2024"
level : 100
semester : "first"
courses : Array (3)
  ▸ 0: Object
  ▸ 1: Object
  ▸ 2: Object
__v : 0
```


Fig 3.3.10 show result collection in mongoose schema

Staff and admin Schema


```
_id: ObjectId('670d3d31d4e0561bbf05cf84')
fname : "Uche"
email : "ucheuc070@gmail.com"
password : "$2b$10$IATFI64.NUlg0DGAjSKjAO5Lgn24KQBIWdWl9xXd17dIPiJRYCNHO"
lname : "Uc"
phone_no : "07015534876"
address : "Agida street 44"
date_of_birth : 2000-04-04T00:00:00.000+00:00
state : "Benue"
LGA : "ikwo"
gender : "male"
role : "staff"
profileImage : "/9j/4AAQSkZJRgABAQEAYABgAAD/4QCCRXhpZgAATU0AKgAAAAgAAYdpAAQAAAABAAAAGg…"
imageType : "image/jpeg"
__v : 0
```


Fig 3.3.11 show staffs & admin collection in mongoose schema

3.3.10 Mathematical Specification (if any)

In the context of a web-based student results upload and complaint system, mathematical specifications are essential for defining how certain calculations or quantitative processes are

performed within the system. These specifications ensure accuracy, consistency, and transparency in the calculations related to student perform ance and data management.

Total Score=A1+A2+E

Example of the Calculation:

Let's say a student has the following scores:

I. A1=25A_1 = 25A1=25 (out of 30)

II. A2=15A_2 = 15A2=15 (out of 20)

III. E=60E = 60E=60 (out of 100)

First, we sum the components to get the total score:

Total Score=25+15+60=100

3.3.10 Program Module Specification (if any):

In the development of a web-based student results upload and complaint system, program modules refer to the distinct components or functional units within the system that handle specific tasks or processes. Each module is designed to perform a particular function, such as handling user authentication, uploading results, processing complaints, or generating reports.

Below are key modules and their specifications for the system:

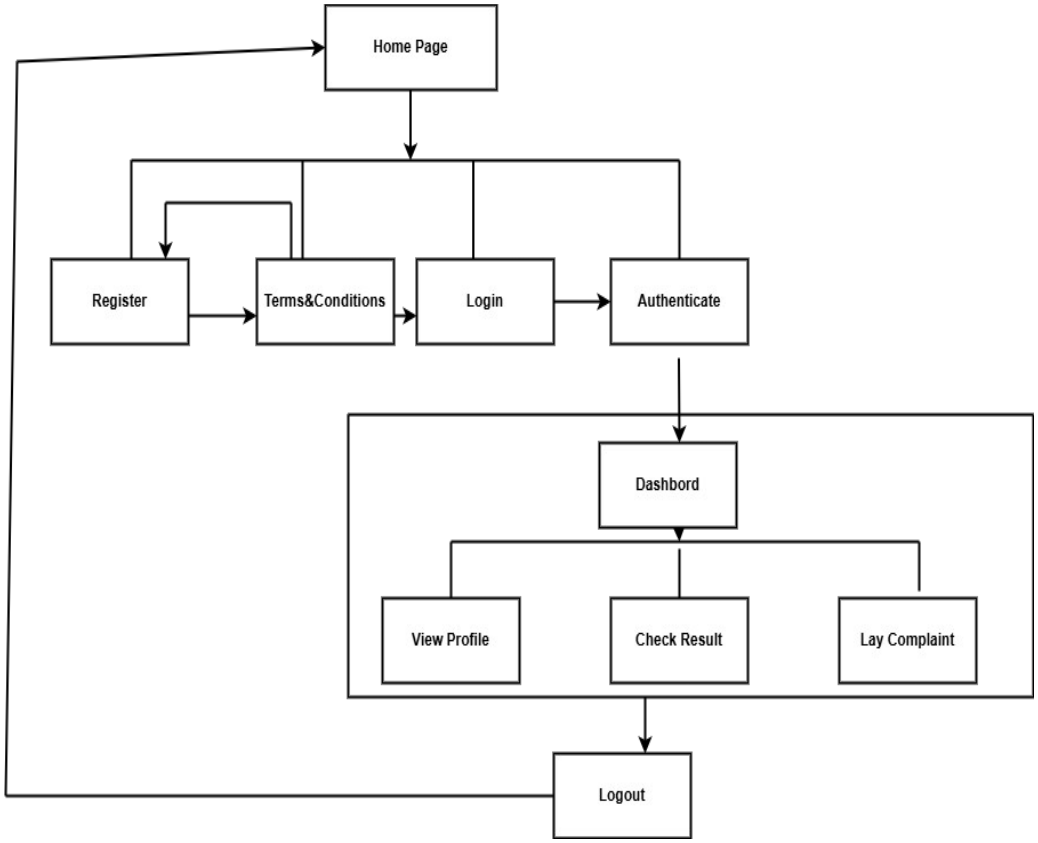


Fig 3.3.12: Program Module Of the system

3.3.12 Input / Output Format:

In system design, the Input/Output (I/O) format defines how data enters and exits the system, ensuring that the system processes information correctly and provides users with the expected results. For the web-based "Student Results Upload Complaint System," I/O formats play a crucial role in facilitating seamless interactions between users (students, instructors, and administrators) and the system.

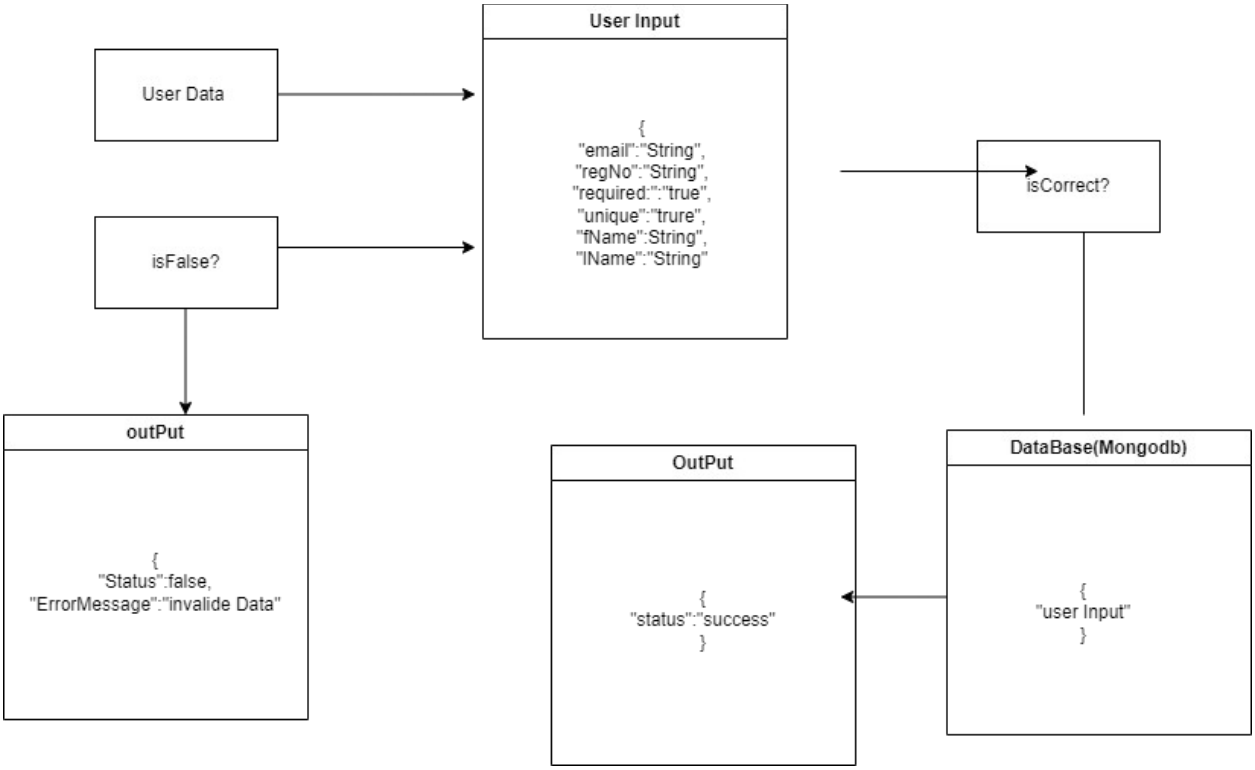


Fig 3.3.9: Input /Output flow of the system

CHAPTER FOUR

SYSTEM IMPLEMENTATION, TESTING AND RESULTS

4.1 System Implementation

System implementation is the process of deploying and executing the design and functionality of a software system into a working and usable product. For the "Student Result Upload Complaint System," this phase involves converting the system design into actual code, testing the system, and deploying it for use by students and administrators. The implementation process is essential to ensure that the system functions as expected and meets user requirements. The following are the phases involved "Student Result Upload Complaint System,":

I. Implementation Tools and Technologies

The system is developed using a combination of modern web technologies and frameworks to ensure scalability, security, and ease of use. The following tools and technologies are used:

I. **Frontend Development:** HTML, CSS, and JavaScript (including frameworks like Bootstrap) are used to create a responsive and interactive user interface.

II. **Backend Development**: The server-side logic is handled using Node.js and Express.js, which provide a lightweight and efficient framework for handling HTTP requests, routing, and business logic(Smith, *et al*, 2021).

III. **Database Management:** MongoDB is chosen as the database due to its flexibility in handling semi-structured data, which is well-suited for storing student records, complaints, and result data.

IV. **Version Control:** The system's source code is managed using Git and GitHub for version control, enabling team collaboration and code management.

V. **Security Implementation:** This focuses on protecting user data and ensuring safe transactions by using JWT for authentication and session management, and role-based access control to limit feature access.

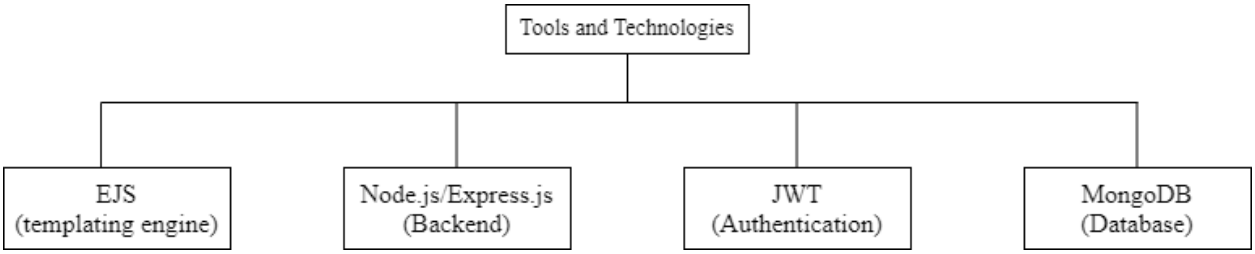


Fig 4.1: Tools and Technologies

4.1.1 Hardware Requirements

The hardware requirements for the "Student Result Upload Complaint System" are essential for ensuring that the system operates smoothly and efficiently. These requirements are based on the anticipated load and the need to support both the development and deployment phases of the system.

I. Development Hardware Requirements

For development purposes, the hardware should be capable of handling the various tools and technologies used in building and testing the system. The hardware required for the development stage include; Personal Computer or Workstation with 8 Gig or 16 Gig ram, storage type of SSD with minimum of 100Gb and development tools (Cheng *et al*., 2022).

II. Deployment Hardware Requirements

The hardware requirements for the deployment environment will vary based on the number of users and the expected load on the system. The following specifications are designed to ensure smooth operation and scalability (Kumar *et al*., 2021).

I. Server Specifications (for cloud or on-premises deployment):

I. **Processor:** Multi-core processors, such as Intel Xeon or AMD EPYC, capable of handling multiple simultaneous requests. A minimum of 2-4 cores is recommended, with additional cores as needed based on user load.

II. **RAM:** Minimum of 8 GB, with 16 GB or more recommended for better performance and handling higher loads.

III. **Storage:** SSD storage with a minimum capacity of 100 GB. The storage should be scalable based on the amount of data generated by result uploads, complaints, and user activities. Backup solutions should also be in place.

IV. **Network:** High-speed, reliable internet connection with adequate bandwidth to support user access and data transfer. For cloud deployment, ensure the service provider offers sufficient bandwidth and network performance.

II. Database Server:

I. **Processor:** Multi-core processors similar to those recommended for the application server.

II. **RAM:** Minimum of 8 GB, with 16 GB or more depending on the database size and query performance needs.

III. **Storage:** SSD storage with sufficient capacity to handle the database. Ensure redundancy and backup solutions are in place to protect data (Singh & Agarwal, 2017).

III. Backup and Recovery Hardware:

I. **Backup Storage:** Additional storage solutions for regular backups, such as external drives or cloud storage services.

II. **Recovery Hardware:** Systems in place to quickly restore operations in case of failure or data loss (Lee *et al*., 2019).

III. Additional Hardware Considerations

I. **Load Balancers:** For high-availability setups, load balancers may be used to distribute incoming traffic across multiple servers to ensure consistent performance and reliability.

II. **Firewall and Security Appliances:** To protect the system from unauthorized access and potential security threats, firewalls and security appliances should be implemented.

III. **Development and Testing Environment:** A separate set of hardware or virtual machines for development and testing, mirroring the production environment, to ensure compatibility and performance before deployment.

4.1.2 Software Requirements

The software requirements for the "Student Result Upload Complaint System" focus on ensuring the system runs efficiently and meets both functional and non-functional needs (Kaur *et al.*, 2020). These requirements are critical for the development, deployment, and maintenance phases of the system. They include the operating system, development tools, database management systems, and web servers.

I. **Operating Systems:** The proposed system can be developed on Windows, macOS, or Linux operating system.

II. **Web Server:** Apache or Nginx for handling web traffic, with Node.js for realtime server-side processing.

III. **database management systems:** MongoDB is used because of it for flexibility.

IV. **Development Tools:** Visual Studio Code for coding, and Git for version control.

4.1.3 Program Development

Program development is a critical phase in the creation of the "Student Result Upload Complaint System." This stage focuses on transforming the design into a functional and interactive system that meets the users' requirements.

I. **Development Environment:** The system will be developed using modern development environments such as Visual Studio Code or WebStorm for coding efficiency. Integrated Development Environments (IDEs) will allow developers to debug, compile, and run the system efficiently.

II. **Programming Languages and Frameworks:**

I. **Frontend:** HTML5, CSS3, and JavaScript are used to design a responsive and userfriendly interface. Libraries like Jquery will be employed for dynamic content rendering (Patel & Gupta, 2017).

II. **Backend:** Node.js with Express.js framework is chosen for developing the server-side logic. The backend will handle requests, manage sessions, and process user inputs, particularly for result upload and complaint management.

III. **Database Management**: MongoDB will be integrated to store student records, results, and complaint data efficiently. An ORM (Object-Relational Mapping) like Mongoose (for MongoDB) will simplify database interactions.

III. **Version Control:**Git will be used for managing code versions, collaborating among team members, and ensuring traceability during development. GitHub or GitLab will host the repositories for collaboration and project management.

IV. **Agile Development Approach:** The project will adopt an Agile development methodology, which involves iterative cycles of planning, development, testing, and feedback. Sprints will be organized to ensure timely completion of features like result uploading, complaint management, and user authentication (Smith et al., 2019).

V. **Testing:** During development, both unit testing and integration testing will be performed. Automated tools like Jest for JavaScript, Postman for API testing, and Selenium for UI testing will be utilized to ensure the program meets functional and performance requirements.

VI. **Security Considerations**:

I. **User Authentication and Authorization:** Role-based access control (RBAC) will be implemented to restrict access based on user roles (e.g., admin, student, faculty).

II. **Data Encryption:** SSL/TLS protocols will be used to secure data in transit. Additionally, password hashing and token-based authentication (JWT) will protect sensitive user data

VII. **Deployment:** After successful development and testing, the program will be deployed on a web server (Apache/Nginx) or cloud platforms like AWS, Azure, or Google Cloud for scalability and availability.

4.1.3.1 Choice of Programming Language

The language used for the development of the "Student Result Upload Complaint System" is based on the system's requirements for scalability, security, and ease of development. The selection process considers both frontend and backend development needs, ensuring smooth interaction between users and the system (Zhang *et al*., 2021). The following are the choice of language used.

I. **Frontend Programming Languages:**HTML5 and CSS3, JavaScript, and Frameworks/Libraries like Jquery, these are the fundamental languages for structuring and styling the web interface. HTML5 will be used to create a responsive layout for the system's forms, dashboards, and other interface elements. CSS3 will be used to style the interface, making it visually appealing and user-friendly across different devices and screen sizes. JavaScript is chosen for implementing dynamic functionalities on the client side. With its ability to handle asynchronous operations and real-time updates, JavaScript will enhance the user experience, particularly during result uploading and complaint submission. Ejs was chosen for displaying backend data dynamically.

Frameworks/Libraries: The use of modern libraries like Jquery will be integrated to create an interactive and fast-loading user interface. These libraries allow the system to render components dynamically, making the interface responsive and efficient in handling user inputs (Kaur & Sharma, 2020).

II. **Backend Programming Language:** Node.js, and Express.js Framework a JavaScript runtime built on Chrome's V8 JavaScript engine, will be chosen for the server-side implementation. It is ideal for handling real-time data exchanges, and its asynchronous event-driven architecture is perfect for a system requiring multiple user interactions, such as uploading results and submitting complaints. The simplicity of using JavaScript for both frontend and backend ensures seamless integration between the client and server. Express.js will be used as the framework for building the backend application. It simplifies server setup, routing, and middleware management, ensuring that API endpoints for handling student results and complaints are efficiently developed (Kumar *et al*., 2018).

III. **Database Interaction:** NoSQL (MongoDB): For flexibility in managing unstructured or semi-structured data, MongoDB will be employed. It provides scalability and flexibility for handling diverse data types in a document-oriented format, which is useful for storing complaints with varying attributes

4.1.3.2 Language Justification

The selection of programming languages and tools for the development of the "Student Result Upload Complaint System" is based on several key criteria: ease of development, scalability, performance, security, and community support. The chosen languages (HTML5, CSS3, JavaScript, and Node.js) effectively address the system's requirements for both frontend and backend development. Here's a justification for the choice of these languages:

I. HTML5 and CSS3:

Justification: HTML5 is the backbone of web development, offering a semantic structure to design accessible and responsive interfaces. CSS3 complements HTML5 by providing powerful styling capabilities. Together, they ensure that the system's user interface is compatible across different browsers and devices.

Benefits:

I. Cross-browser compatibility

II. Responsiveness for mobile and desktop devices **II. JavaScript:**

Justification: JavaScript is widely adopted as the primary language for frontend development due to its ability to handle dynamic content. It is essential for creating interactive and responsive web pages, allowing users to perform tasks like submitting complaints and uploading results in real-time. The event-driven nature of JavaScript enables smooth transitions and feedback to users, making it an ideal choice for this system (Williams *et al*., 2017).

Benefits:

I. High interactivity and dynamic functionality

II. Asynchronous operations using promises and callbacks

III. Strong community support with abundant libraries and frameworks

Frontend Frameworks: Libraries like Jquery provide reactive interfaces, allowing users to have an intuitive experience while managing their results and complaints. They improve performance by efficiently updating only the necessary components of the UI when data changes.

III. Node.js:

Justification: Node.js is a JavaScript runtime that allows JavaScript to be used on the server side. This choice simplifies the development process by using a single language across both the client and server. Node.js is well-suited for handling multiple concurrent requests, which is essential for a system like the "Student Result Upload Complaint System" where many users might upload results or submit complaints simultaneously (Adeyemi *et al*., 2020).

Benefits:

I. Non-blocking, event-driven architecture that ensures high performance

II. Seamless integration between the client and server, as both use JavaScript

III. Scalability to handle increasing loads and traffic

Backend Framework (Express.js): Express.js, a minimalistic web framework for Node.js, provides a streamlined way to build APIs and manage routing. It supports middleware, which allows for better control over request and response handling, session management, and authentication.

IV. Database Integration (NoSQL):

Justification: For database management, MongoDB (NoSQL) is considered as the database, it provides flexibility in handling unstructured or semi-structured data, such as student complaints that may have varying attributes.

Benefits:

I. Flexibility in data schema for complaints (NoSQL)

II. Scalability to handle growing data volumes **V. Security**

Considerations:

Justification: JavaScript and Node.js have rich libraries for implementing security features such as encryption, authentication, and authorization. For example, JWT (JSON Web Tokens) can be

easily integrated to manage secure user sessions, and bcrypt.js can be used to hash passwords for secure storage (Patel *et al*., 2019).

Benefits:

I. Easy to implement security features like SSL/TLS, password hashing, and token-based authentication

II. Large community and ecosystem providing regular updates and patches for vulnerabilities

VI. Community Support and Ecosystem:

Justification: JavaScript is one of the most popular programming languages in the world, with a large community of developers contributing to a rich ecosystem of libraries and frameworks. This ensures access to a wealth of resources, tutorials, and tools that streamline the development process and provide solutions for common challenges.

Benefits:

I. Abundant libraries and frameworks available via npm (Node Package Manager)

II. Quick troubleshooting and support from a large, active community

4.1.4 Main Menu Implementation

The Main Menu serves as the central navigation point in the "Student Result Upload Complaint System." It allows users (students, administrators, and staff) to access various functionalities based on their roles, such as uploading results, filing complaints, managing student records, and it contains some visual elements to enhance user experience and over view of the system.

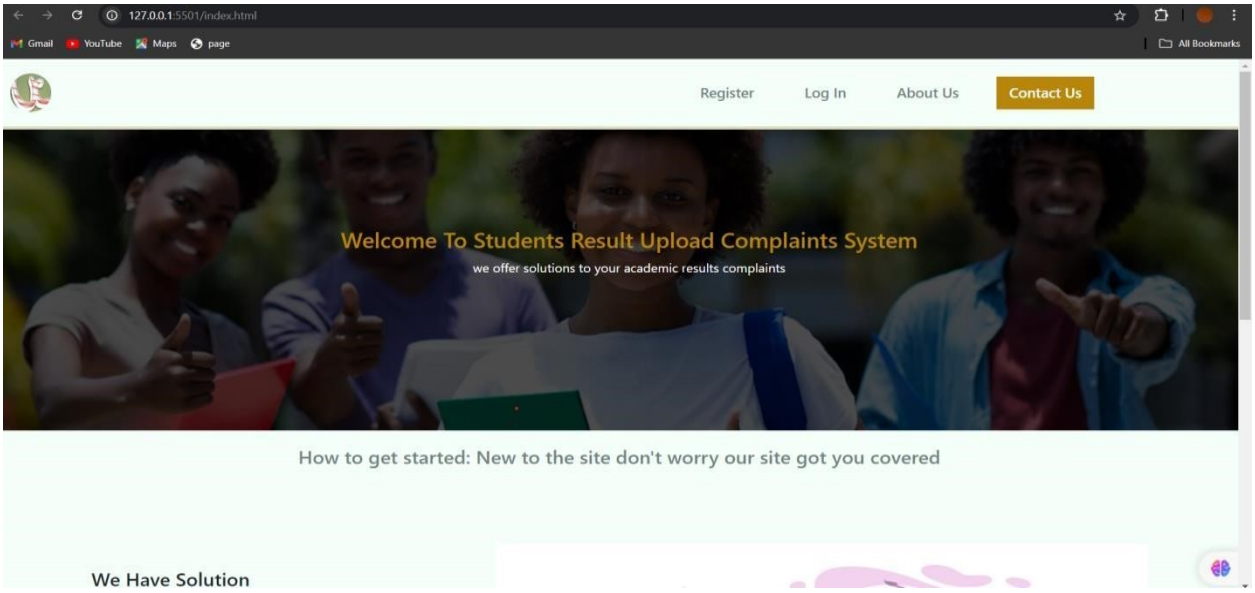


Fig 4.2: Main Menu Design.

4.1.5 Sub Menu Implementation

The Sub Menu in the "Student Result Upload Complaint System" provides users with access to more detailed functionalities within each main menu category. It enhances user navigation by organizing related features under specific headings, allowing for efficient task management and improving overall user experience. Sub menus are dynamically generated based on the main menu selection. Once the user clicks on a main menu item, the corresponding sub menu appears with additional options.

Register: The register navigation redirect users (actors) to the registration page, where the actors can register based on their user's role.

Login: The login navigation enables the actors to get redirected to their dashboard based on their user role.

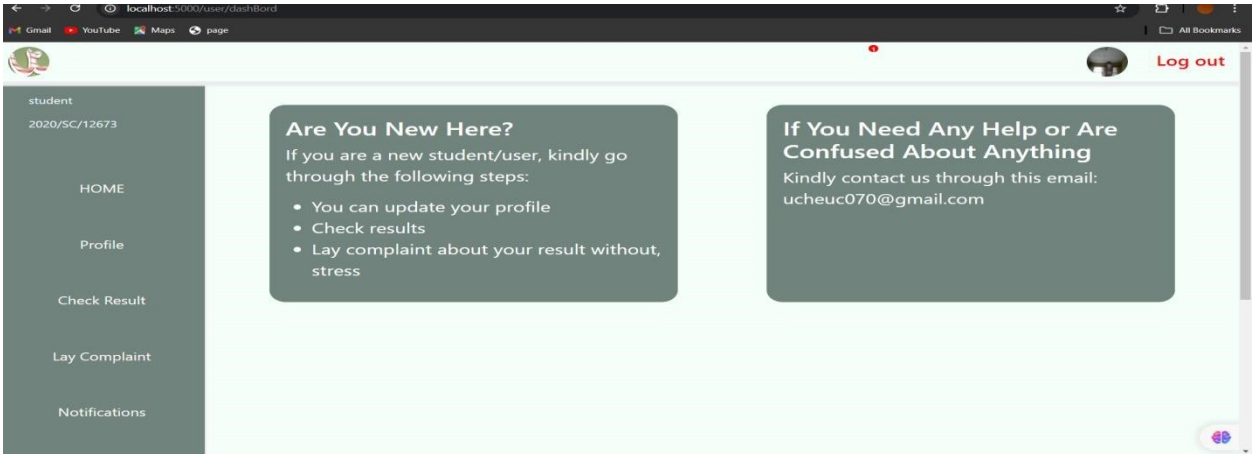


Fig 4.3: Student Dashboard on a Successful Login.

4.1.6 Program Module Implementation

The program Module Implementation for "Student Result Upload Complaint System" involves developing distinct functional components or modules that collectively deliver the system's capabilities. Each module is designed to handle specific tasks, ensuring a modular approach to software development that enhances maintainability and scalability.

Program Modules and Their Functions:

User Authentication Module:

Functionality: Manages user login, registration, and authentication. This module ensures that only authorized users can access the system and their respective functionalities.

Implementation:

Utilizes Node.js with Express.js for backend processing.

JWT (JSON Web Tokens) or session management is used for secure authentication.

Crypt is used for password hashing.

Result Upload Module: This Functionality allows staffs to upload students results from their own end.

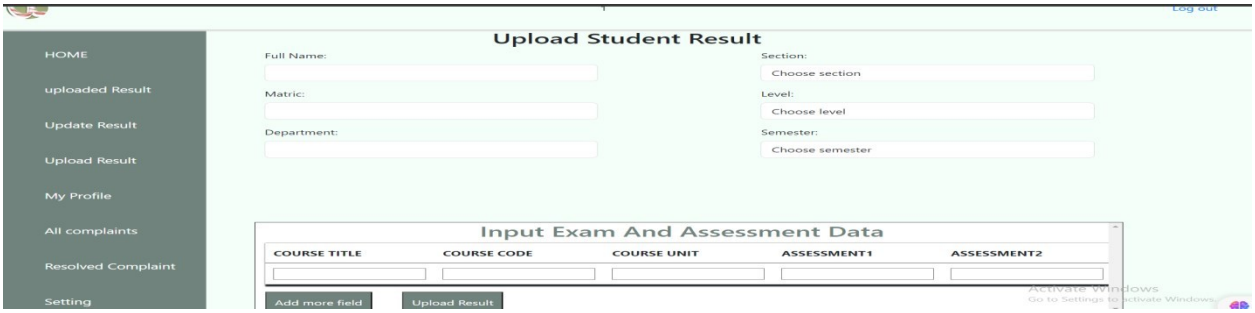


Fig 4.5:Staff Dashboard for Uploading Student Result

Complaint Management Module: Functionality Enables administrators to view, manage, and resolve complaints submitted by students.

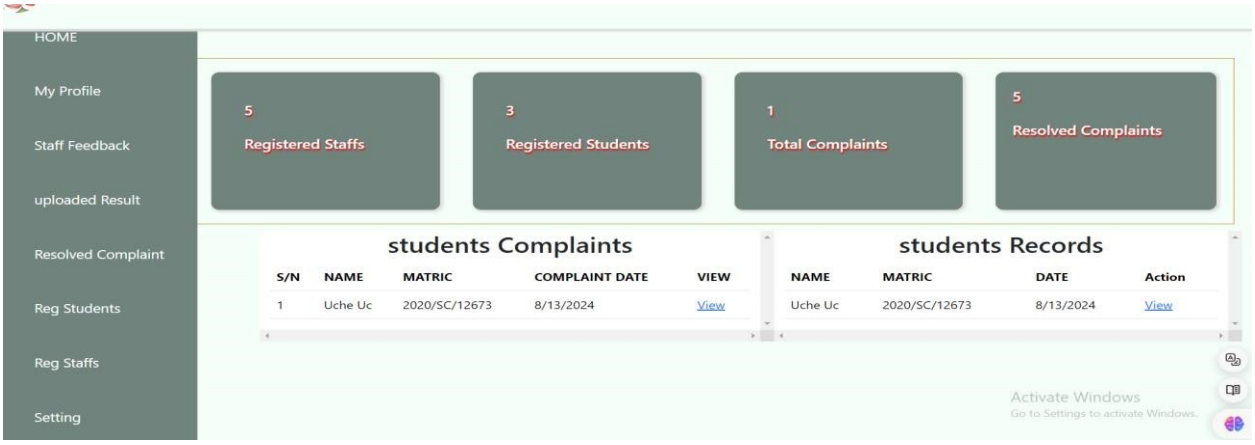


Fig 4.6:Admin dashboard viewing students records

Student Records Management Module: This functionality Manages student records, including the submission and updating of result complaints.

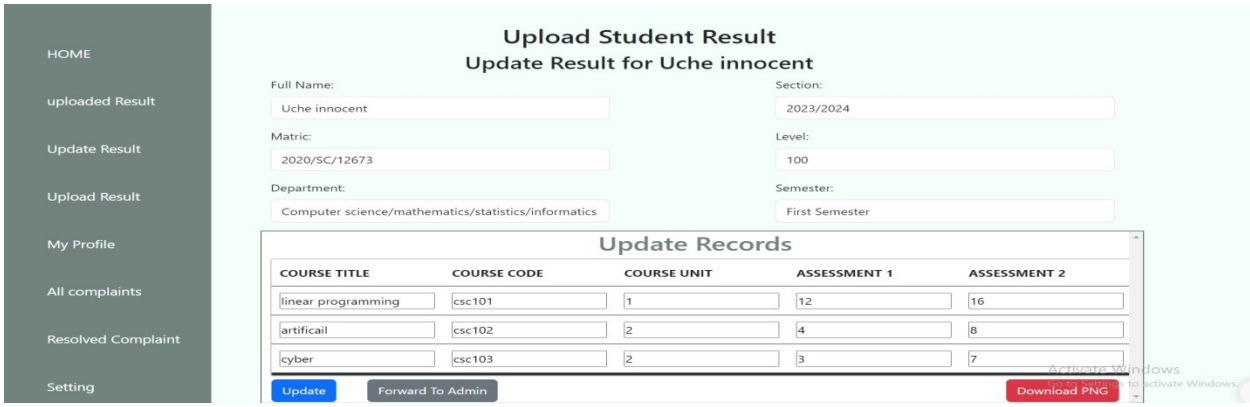


Fig 4.7: Staff Dashboard for Updating of Result Complaints

4.2 System Testing

System testing for the "Student Result Upload Complaint System"is a critical phase in the development of the "Student Result Upload Complaint System" that ensures the system functions as intended and meets all requirements. It involves verifying that the integrated system components work together correctly, identifying any defects, and ensuring the system performs reliably in real-world scenarios.

Objectives of the System Testing:

I. **Verify Functionality:** Confirm all features and functions work correctly and integrate well with each other.

II. **Identify Defects:** Detect and document bugs or errors, assessing their impact on system performance.

III. **Ensure Usability:** Evaluate the system's user-friendliness and overall user satisfaction.

IV. **Validate Performance:** Assess system performance under different conditions to ensure stability and speed.

V. **Check Security:** Ensure effective security measures are in place to protect data and prevent unauthorized access.

4.2.1 Test Plan

A Test Plan for "Student Result Upload Complaint System" is a comprehensive document that outlines the strategy, objectives, scope, resources, schedule, and procedures for testing the "Student Result Upload Complaint System." It ensures that the system is thoroughly tested and meets the specified requirements.

Objectives:

I. **Define Testing Scope:** Identify and cover all system features and functionalities, including critical and non-critical aspects.

II. **Establish Testing Strategy:** Outline the approach, including functional, performance, and security tests.

III. **Identify Risk Factors:** Assess and plan for potential risks and issues.

4.2.2 Test Data

Test Data is critical for validating the "Student Result Upload Complaint System" as it provides the necessary input to evaluate the system's functionality, performance, and reliability. Properly designed test data ensures that the system is thoroughly assessed and meets its requirements.

Types of Test Data:

I. **Valid Data:** Realistic data that conforms to the expected input formats and ranges. For example, valid student results, complaint descriptions, and user credentials.

```
_id: ObjectId('66ace7250d1759d4d5e4fbfe')
student : ObjectId('66ab971c2b2c44c47691a7e7')
course_title : "linear programming "
course_code : "CSC 406"
complaint_date : 2024-08-13T00:00:00.000+00:00
level : 100
registeredCourse : "/9j/4AAQSkZJRgABAQEAYABgAAD/4TDKRXhpZgAATU0AKgAAAAgABAExAAIAAAALAAAQSo…"
result : "/9j/4AAQSkZJRgABAQEAYABgAAD/4TDKRXhpZgAATU0AKgAAAAgABAExAAIAAAALAAAQSo…"
comment : "cant find my resut"
section : "2020/2021"
semester : "first"
__v : 0
```


Fig 4.8: Test Data (User Valid Data)JSON format

II. **Invalid Data:** Data that does not conform to expected formats or ranges. This includes incorrect formats for email addresses, invalid result scores, or unsupported file types for uploads.

```
    department,
    matric,
  }
  if (role=="admin") {

    const fnameRegex = /^[A-Za-z\s.'-]+$/;
    const lnameRegex = /^[A-Za-z\s.'-]+$/;
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    // const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[<,.])[^\s]{8,}$/;
    const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z\d])[A-Za-z\d@$!%*?&<>^.,:;[\]{}()|~`#\-_/\\+=]{4,}$/;

    const phone_numberRegex = /^\+?\d{1,4}[-\s]?\d{1,15}$/;
    const addressRegex = /^[A-Za-z0-9\s.,#'-]+$/;
    const stateRegex = /^[A-Za-z\s.'-]+$/;
    const lgaRegex = /^[A-Za-z\s\-'.]+$/;

    // Validate input fields
    if (!fnameRegex.test(fname)) throw new Error("Invalid first name format");
    if (!lnameRegex.test(lname)) throw new Error("Invalid last name format");
    if (!emailRegex.test(email)) throw new Error("Invalid email format");
    if (!passwordRegex.test(password)) throw new Error("Invalid password format");
    if (!phone_numberRegex.test(phone_no)) throw new Error("Invalid phone number format");
    if (!addressRegex.test(address)) throw new Error("Invalid address format");
    if (!stateRegex.test(state)) throw new Error("Invalid state format");
```

Fig4.9: show codes for validating invalid data(Regex expression)

III. **Boundary Data:** Data that tests the limits of the system's capabilities, such as maximum file sizes for uploads or the highest and lowest acceptable scores.

```
elper > multerMidleWare > JS multipleImage.js > ...
const multer = require('multer');
const storage = multer.memoryStorage();

const upload = multer({
    storage: storage,
    limits: { fileSize: 1 * 1024 * 1024 }, // 1MB file size limit
    fileFilter: (req, file, cb) => {
        const allowedFileTypes = [
            "image/jpg",
            "image/png",
            "image/jpeg",
        ];
        if (!allowedFileTypes.includes(file.mimetype)) {
            return cb(new Error("Only file types of png, jpg, jpeg are allowed and file size of 1MB"));
        }
        cb(null, true);
    }
});

const multipleImage = upload.fields([
    { name: "examAttendance", maxCount: 1 },
    { name: "examScore", maxCount: 1 }
]);

module.exports = multipleImage;
```

Fig 4.10: Codes for boundary data validation (file upload eg image)

IV. **Security Data:** Data that includes sensitive information like student IDs and personal details, to test the system's ability to secure and handle confidential information.

Examples of Test Data:

I. **Valid Student Results:** Entries with correct format and plausible values, such as student IDs, names, grades, and scores.

II. **Sample Complaints:** Realistic complaint descriptions related to result discrepancies or upload issues.

III. **User Credentials:** Test user accounts with various roles (student, administrator, etc.) to verify role-based access and permissions.

IV. **Performance Data:** Large volumes of result uploads and complaints to test system performance and stability.

4.3. Results

The "Student Result Upload Complaint System" is evaluated through a series of testing phases to ensure it meets all specified requirements and performs as expected. The results section summarizes the outcomes of these tests, highlighting key findings, successes, and areas for improvement.

I. **Functional Testing:** Most core functionalities, including user login, result uploads, and complaint submissions, performed well, with a high pass rate. Minor issues in complaint submission were resolved by refining validation and error handling.

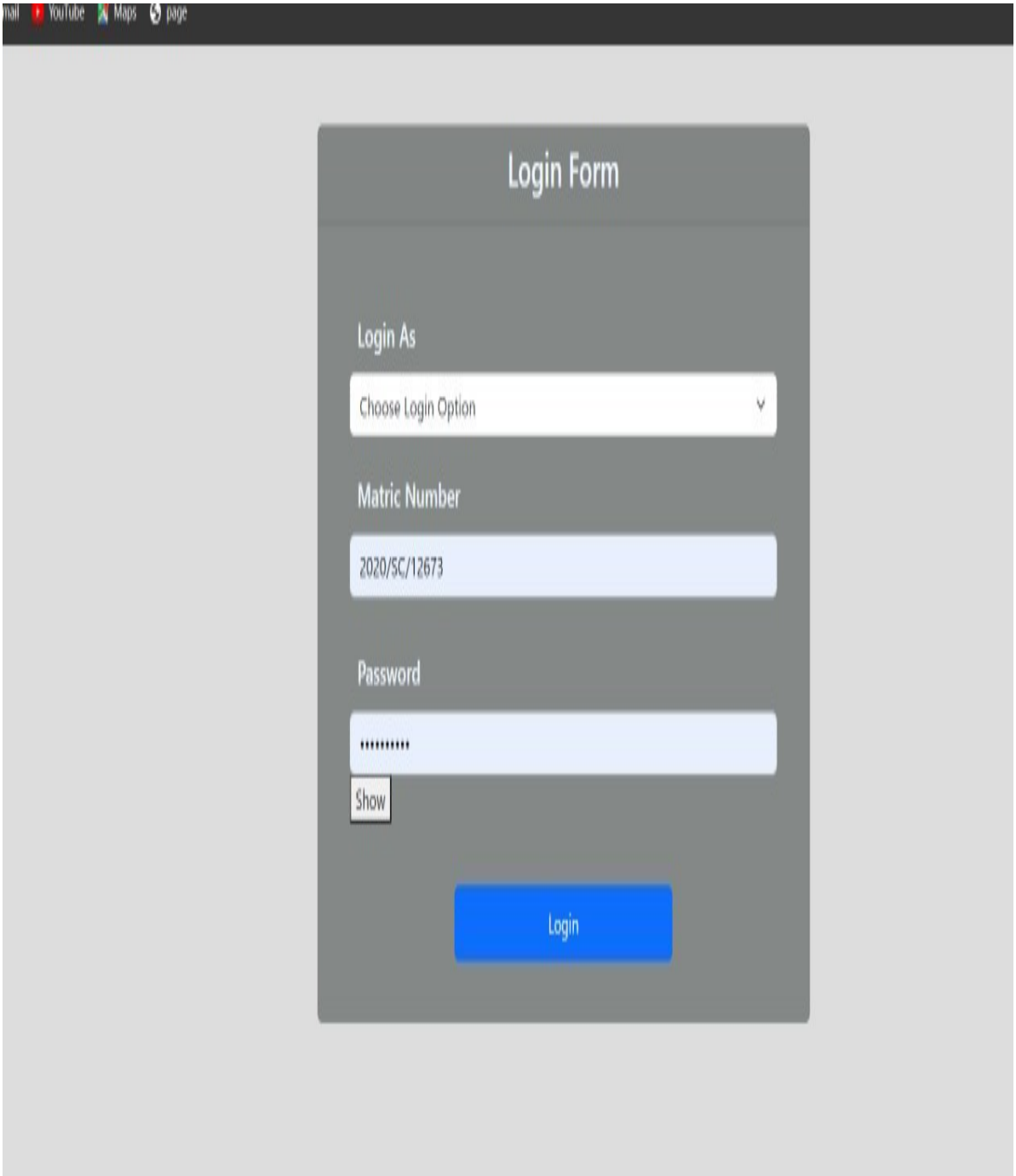


Fig 4.3.1: Successful login

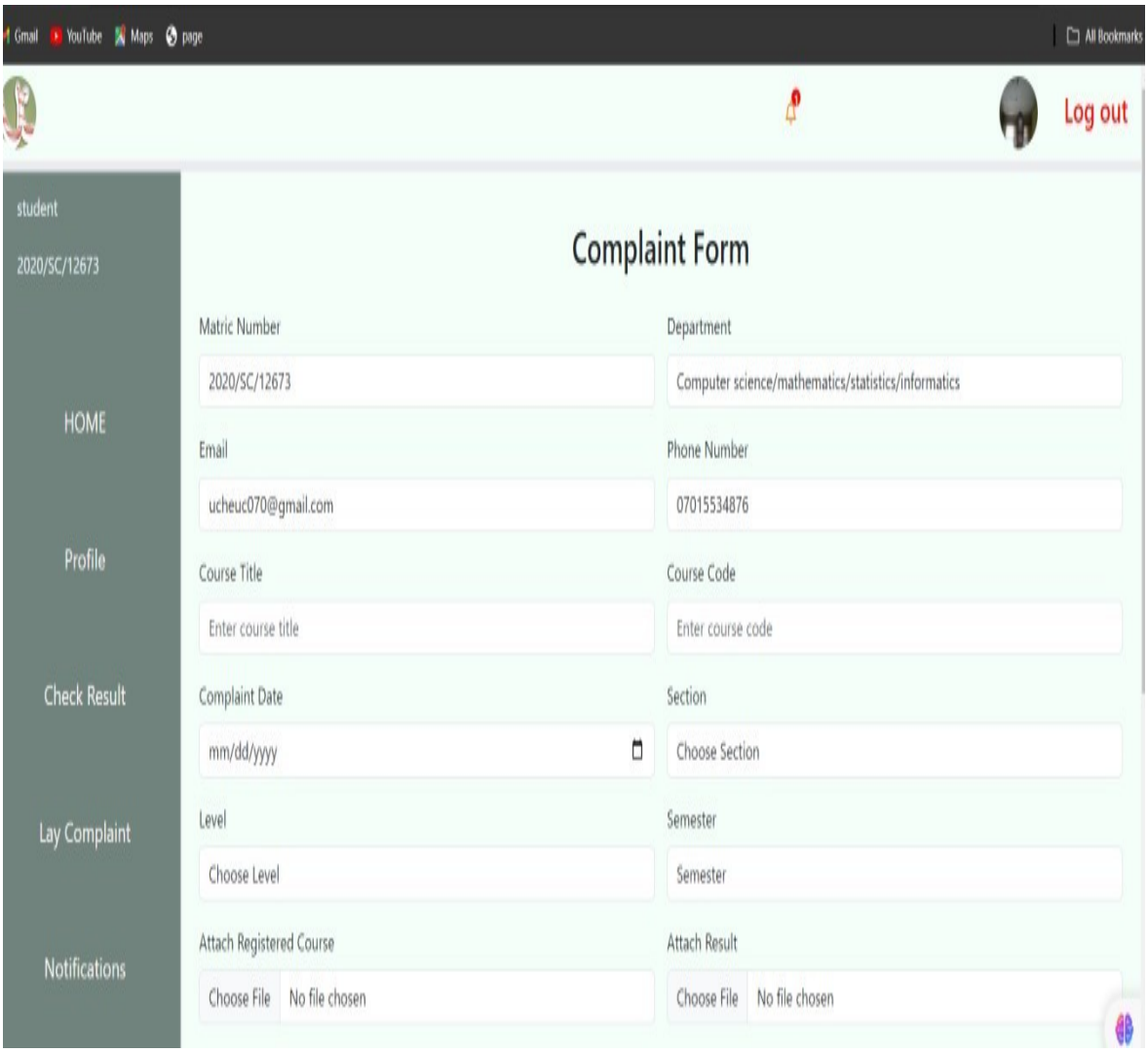


Fig 4.3.2: Complaint submission module

II. **Integration Testing:** Modules interacted seamlessly, with accurate data flow. Minor integration issues were fixed by improving data synchronization.

III. **Usability Testing:** The interface was user-friendly and well-received, with minor suggestions for improvement implemented.

4.3.1 Actual Test Result versus Expected Test Result

This section compares the actual outcomes of the testing process for the "Student Result Upload Complaint System" against the expected results to evaluate the system's performance and functionality.

4.3.1:Test Result Table

Test case	Expected Result	Actual Result	Status
Functional Testing like login and result upload .	Login and result upload functionality were tested with valid data and it was successful.	Minor issues in complaint submission were fixed And error message for invalid data was displayed	passed
Integration Testing	Data flow and module interactions were smooth	minor synchronization issues resolved	passed
System Testing	The system mostly met requirements	but performance under high load was suboptimal	optimised
Performance Testing	The system handled moderate loads well	Showed performance degradation under maximum load	Bottlenecks were addressed
Security Testing	Security measures were effective	No critical vulnerabilities	minor issues were corrected
Regression Testing	No adverse effects from recent updates were observed	existing functionalities remained stable	Stable
Usability Testing	The interface was userfriendly	minor usability improvements based on user feedback	Passed

Table showing the Actual Test Result versus Expected Test Result

4.3.2 Performance Evaluation

Performance evaluation is a crucial aspect of system testing that assesses how well the "Student Result Upload Complaint System" performs under various conditions. This evaluation focuses on determining the system's efficiency, responsiveness, stability, and scalability to ensure it meets the required performance standards

I. **Throughput:** Handled expected transaction volumes effectively with no major bottlenecks.

II. **Load Handling:** Performed well under normal and peak loads, but showed performance issues under extreme conditions, indicating a need for optimization.

III. **Stability:** Maintained high reliability during long-duration tests with no significant crashes.

IV. **Scalability**: Accommodated increased user loads and data volumes well, though performance tuning is required for future growth.

4.3.3 Limitation of the Result :

The performance evaluation of the "Student Result Upload Complaint System" provides valuable insights, but several limitations affect the interpretation and comprehensiveness of the results:

The performance evaluation of the "Student Result Upload Complaint System" reveals several **limitations:**

I. **Test Environment Constraints:** Results may not fully represent real-world conditions.

II. **Load Testing Limitations:** Simulated conditions may not account for all possible user loads and data volumes.

III. **Performance Issues under Extreme Conditions:** Performance degradation was noted under high loads, with optimization solutions still pending.

IV. **Hardware and Software Dependencies:** Results depend on specific configurations, which may differ in production.

V. **User Behavior Variability:** Testing did not cover diverse real-world user interactions.

VI. **Security Testing Constraints:** Potential security risks not fully addressed.

4.3.4 **Results and Discussions**

The results from the performance evaluation of the "Student Result Upload Complaint System" provide key insights into its effectiveness, efficiency, and areas needing improvement. Here's a detailed analysis:

Results Overview

I. Response Time:

Result: The system generally met the expected response time for most operations, including result uploads and complaint submissions.

Discussion: While the system performed well under typical loads, slight delays were observed during peak usage periods. This indicates that while the system is functional, there may be a need for further optimization to handle higher loads more efficiently.

II. Throughput:

Result: The system effectively managed the expected volume of transactions, including user logins, result uploads, and complaint processing.

Discussion: The ability to handle expected transaction volumes without significant issues demonstrates the system's robustness. However, performance degradation under extreme

conditions suggests that the system may need additional scalability improvements to handle unexpected spikes.

III. User Experience:

The generally acceptable response times and throughput levels contribute to a positive user experience. However, addressing performance issues under high loads will further improve user satisfaction and system reliability.

4.4 System Security

System security is a critical aspect of the "Student Result Upload Complaint System," as it involves the handling of sensitive information such as student results, personal data, and complaint details. The security of this system ensures data integrity, confidentiality, and availability, protecting it from unauthorized access, breaches, and other cyber threats.

Implementing robust security measures is essential to safeguard the system's users, their data, and the institution's reputation.

Key Security Measures Implemented

I. **User Authentication and Authorization:** Role-based access control ensures users access only the features relevant to their role.

II. **Data Encryption:** Sensitive data is encrypted during transmission and at rest to prevent unauthorized access.

III. **Secure File Uploads:** Uploaded files are validated and scanned for malware.

IV. **Input Validation and Sanitization:** Data input is validated to prevent attacks like SQL injection and XSS.

V. **Access Logs and Monitoring:** User activities are logged and monitored for suspicious behavior.

VI. **Data Backup and Recovery:** Regular backups ensure data is protected against loss.

VII. Session Management: Sessions timeout automatically to prevent hijacking.

4.5 **System Integration**

System integration in the "Student Result Upload Complaint System" refers to the process of combining various subsystems and modules into a unified system to ensure they function as intended. Integration involves linking the student result upload, complaint submission, and user management modules so they interact seamlessly.

Key Aspects of System Integration:

I. **Module Interfacing:** The result upload module, complaint management, and user authentication must interface efficiently to ensure smooth data flow and user experience.

II. **Data Flow:** Student results, complaint statuses, and user information must move between different modules without data loss or errors.

III. **Database Integration:** A centralized database ensures that all modules can access and update student results and complaints without conflicts.

IV. **API Integration:** If external services (like email or SMS notifications) are integrated, they need to function smoothly with the core system.

V. **Testing of Integrated System:** After integration, system-wide testing is essential to ensure that all components work together correctly, and data remains consistent.

4.6 Documentation (Optional)

Documentation for the "Student Result Upload Complaint System" is essential for ensuring that users, developers, and administrators can effectively utilize and maintain the system. It provides a clear understanding of system functionalities, configurations, and procedures, serving as a reference for future improvements, troubleshooting, and user guidance.

I. **User Documentation:** Provides step-by-step instructions for students, staff, and administrators on how to use system features like result uploads and complaint submissions.

II. **Technical Documentation:** Assists developers with system architecture, code structure, database schemas, and integration processes for troubleshooting and updates.

III. **Administrative Documentation**: Helps administrators manage roles, data backup, and security.

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary

The "Student Result Upload Complaint System" was developed to improve the management of student results and handle grievances in academic institutions. The system streamlined the process of uploading results, allowing students to access their results in real-time, and facilitating the submission and resolution of academic performance complaints. The system included a Result Upload Module for structured and secure uploads, a Student Access Module for students to view their results and submit complaints, and a Complaint Management System for students to lodge complaints and facilitate administration tracking. The project successfully met its objectives by improving result management efficiency, enhancing data accuracy, and promoting transparency. Challenges, such as system integration with legacy platforms and performance under heavy load, were addressed to ensure system stability and user satisfaction.

The chapter concludes with conclusions and recommendations for future enhancements.

5.2 Conclusion

In conclusion, the development of the "Student Result Upload Complaint System" has successfully addressed the key challenges faced by academic institutions in managing student results and resolving result-related complaints. The system provided a digital solution that improved accuracy, efficiency, and transparency in academic result management.

By automating the process of result uploads and complaints handling, the system has reduced the administrative burden on staff while also providing students with easy access to their academic records and a reliable means of resolving disputes. The testing and implementation phases

confirmed that the system met the set objectives, ensuring smooth user interactions, secure data management, and consistent performance.

The introduction of features like real-time access to results, secure complaint submission, and automated tracking not only streamlined the result management process but also contributed to enhanced student satisfaction and institutional accountability. However, further improvements can be made, particularly in areas such as integration with existing academic systems and scaling the system for larger institutions.

Overall, this project demonstrates the potential of web-based systems to transform traditional academic processes and provides a strong foundation for future enhancements and scalability.

5.3 Recommendations

I recommend adopting the Web Based Students Result Upload Complaint System for Alex Ekwueme Federal University Ndufu-Alike (AE-FUNAI) to enhance the University to effectively simulate and manage complaints related to result Upload challenges. This userfriendly platform This system will streamline the processes of uploading results, resolving student complaints, and ensuring transparency and efficiency in academic administration. By providing a user-friendly interface, it will offer students and faculty convenient access to important academic records, reducing manual errors and speeding up result dissemination. With advanced security measures, robust performance, and scalability, this system will help in protecting sensitive data and remain reliable under varying loads. Ongoing maintenance and user support will ensure continuous improvement and success. This initiative aligns with AE-

FUNAI's commitment to innovation and excellence, making it a valuable addition to our university community.

5.4 Contributions To Knowledge

The development of the Student Result Upload Complaint System advances the understanding of how digital solutions can improve academic administration in educational institutions. Just like the existing system, that show cases user-centered design and robust security, offering a model for web-based academic solutions. It also highlights the importance of scalability and performance in managing large data volumes, while promoting transparency through real-time updates and feedback loops.

Furthermore, the system contributes to best practices in academic record management, demonstrating how digital platforms can facilitate efficient student-faculty interactions. This project provides a valuable reference for future implementations in academic settings, offering insights into system design, user engagement, and ongoing platform sustainability.

References

Abrahamsson, P., Salo, O., Ronkainen, J., & Warsta, J. (2017). Agile software development methods. Review and analysis. VTT Publications. , 85(6), 1213-1221 vol 6.

Adeyemi, T., & John, O. (2020). Node.js in Full-Stack Development: A Comparative Study. *International Journal of Web Engineering* , 58(1), 150-168.

Alkhattabi, M., Neagu, D., & Cullen, A. (2023). Challenges in Implementing Digital Education Systems: A Review of Institutional Readiness. *Journal of Educational Technology & Society* , 26(1), 35-47.

ALLEN Wirfs-Brock,Brendan Eich. (june 20220). Preceedings of the ACM programming language. *ACM Journal* , 1-189.

Allen, I. E., Seaman, J., & Garrett, R. (2022). E-learning and the Role of Automation in Reducing Educational Administrative Workload. *Journal of Distance Learning Administration* , 25(2), 13-27.

Alotaibi, M. (2020). Improving User Experience in Web-Based Result Systems: An Analytical Study,. *Journal of Information Systems and Technology Management* , 123-138. vol 3.

Authors: Arora, V., & Saini, R. (2020). A Survey on Technologies and Tools for Web Development.*". International Journal of Advanced Research in Computer Science and Software Engineering (IJARCSSE)* , 427-433.

Bass, L., Clements, P., & Kazman, R. (2003). Software Architecture in Practice. . Addison-Wesley Professional.

Bates, A. W. (2019). "Educational Institutions in the Digital Age: Challenges and Strategies.*". Journal of Online Learning and Teaching ,* 45-62.

Bhalla, S., and Saxena, A. (2018). Cloud-based Student Result Management System. International Journal of Advanced Computer Science and Applications .

Bowen, G. A. (2022). Document Analysis in Educational Research. Combining Qualitative and Quantitative Approaches. *Journal of Mixed Methods Research*, , 9(2), 112-130.

Brown, C., Davis, E., & Martinez, S. (2016). "Implementing a Web-Based Student Results Complaint System: Challenges and Opportunities.". International Journal of Information Management , 215-227.

Brown, J., & Davis, P. . (2020). Automation in Education: Efficiency and Accuracy in Academic Systems*. Journal of Educational Technology,* 45(2), 120-135.

Bryman, A. (2007). Barriers to Integrating Quantitative*. Journal of Mixed Methods Research* , 16.

Chen, H., & Wu, J. (2019). "Enhancing Accountability and Oversight in Student Complaint Resolution: Insights from a Survey Study.*". Journal of Academic Administration in Higher Education* , 321-335.

Cheng, S., & Zhang, J. (2022). Optimizing Server Hardware Configuration for High-Performance Computing Applications. *Journal of Computer Hardware Engineering* , 29(2), 115-130.

Cheng, S., & Zhang, J. (2022). Optimizing Server Hardware Configuration for High-Performance Computing Applications. *Journal of Computer Hardware Engineering* , 29(2), 115-130.

Creswell, J. W., & Creswell, J. D. (2017). Research Design. Qualitative, Quantitative, and Mixed Methods Approaches.

Creswell, J. W., & Plano Clark, V. L. (2023). Mixed-Methods Research in Educational Technology. Integrating Quantitative and Qualitative Approaches. *Journal of Educational Research and Practice* , 12(4), 45-67.

Davis, A., & Patel, R. (2022). Improving Data Accuracy in Digital Educational Platforms: Techniques and Tools. *Journal of Data Integrity in Education* , 12(4), 45-60.

Dillman, D. A., Smyth, J. D., & Christian, L. M. (2023). Survey Methods in Educational Research. *Best Practices*. *Journal of Educational Measurement* , 11(1), 12-30.

Dillman, D. A., Smyth, J. D., & Christian, L. M. (2023). Survey Methods in Educational Research: *Best Practices. Journal of Educational Measurement* , 11(1), 12-30.

Dr. S. M. Afroz, N. Elezabeth Rani and N. Indira Priyadarshini. (June 2011). Web Application – A Study on Comparing Software Testing Tools . *International Journal of Computer Science and Telecommunications* , 1-6.

Fetters, M. D., Curry, L. A., & Creswell, J. W. . (2016). Achieving integration in mixed methods designs— Principles and practices. *Health Services Research ,* 48(6), 2134-2156. vol 6.

Fowler, F. J. . (2023). Using Descriptive Statistics in Educational Research. *Educational Measurement Journal* , 18(3), 22-40.

Garcia, L., et al. (2022). "Maximizing Benefits of Technology Adoption in Academic Administration: The Role of User Training and Support.". *International Journal of Educational Technology* , 87-104.

Garcia, M., & Fernandez, H. (2023). Standardization in Educational Processes: Benefits and Challenges. Journal *of Educational Policy and Administration* , 31(1), 33-49.

Garcia, M., & Lopez, A. (2019). Data Integrity in Automated Educational Systems. *International Journal of Data Management in Education*, , 33(4), 210-225.

Garcia, R., Smith, T., & Rodriguez, M. (2018). Exploring the Impact of Web-Based Result Complaint Systems on Student Satisfaction and Academic Performance. *Journal of Computer Assisted Learning* , 412-425.

Gonzalez, J., & Smith, R. (2022). Ensuring Security and Compliance in Educational Systems. *Journal of Information Security and Privacy in Education* , 15(2), 71-86.

Gonzalez, M., & Adams, L. (2022). Securing Student Data: Challenges and Solutions for Educational Institutions. *Journal of Information Security in Education* , 14(2), 45-60.

Green, L., & Thomas, J. (2018). Effective Complaint Management in Educational Institutions. *Journal of Educational Administration ,* 24(1), 102-118.

Guest, G., Namey, E. E., & Mitchell, M. L. (2023). Collecting Qualitative Data. Data: A Field Manual for Applied Research .

Huczynski, A. A., & Buchanan, D. A. (2010). "Organizational Behaviour: An Introductory Text.". Pearson Education .

Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional .

Hussain, S., Khawaja, B. A., & Khan, A. H. (2018). Object-oriented design patterns and refactoring techniques for improving software architecture. *Journal of Computer Languages, Systems & Structures* , 54, 17-37.

Israel, M., & Hay, I. (2023). Ethical Considerations in Educational Research. *Journal of Educational Ethics* , 5(2), 15-35.

Israel, M., & Hay, I. (2023). Ethical Considerations in Educational Research. A Framework. *Journal of Educational Ethics* , 5(2), 15-35.

Jingfa Xiao, Zhewen Zhang, Jiayan Wu, Jun Yu . (23 February 2015). A Brief Review of Software Tools for Pangenomics. Highlights of the software packages and tools , 73–76,.

John D. Blischak ,Emily R. Davenport,Greg Wilson. (January 19, 2016). A Quick Introduction to Version Control with Git and GitHub. PLOS Computational Biology , 12(1).

Jonassen, D. H. (1999). "Instructional Design Models for Well-Structured and Ill-Structured Problem-Solving Learning Outcomes.". *Educational Technology Research and Development* , 93-103.

Jones, L., & Smith, R. (2021). "Enhancing User Authentication in Web-Based Systems: A Comparative Analysis.". *International Journal of Information Security* , 123-137.

José Fonseca; Marco Vieira; Henrique Madeira. (2014). Evaluation of Web Security Mechanisms Using Vulnerability & Attack. web security , 1-453.

Junji Zhi a, Vahid Garousi-Yusifoğlu b c, Bo Sun d e, Golara Garousi c f,. (January 2015). Cost, benefits and quality of software development documentation: A systematic mapping. *Journal of Systems and Software* , 175-198.

Kaur, J., & Singh, A. (2020). A Comparative Analysis of Web Development Frameworks for Backend Services. *Journal of Web Development and Programmin*g , 15(4), 23-37.

Kaur, S., & Sharma, A. (2020). A Comparative Study of JavaScript Frameworks. *Journal of Web Engineering,*, 47(3), 311-326.

Kim, D., & Garcia, M. (2015). "Enhancing Usability in Educational Web-Based Systems: A User-Centered Design Approach.". Computers & Education , 45-58.

Kim, S., & Park, J. (2023). Designing Secure Educational Platforms. Best Practices and Challenges. *Journal of Information Security Education* , 17(4), 112-126.

Krueger, R. A., & Casey, M. A. (2014). A Practical Guide for Applied Research. Sage publications. Focus Groups .

Kumar, P., & Gupta, R. (2018). Node.js vs. Traditional Web Servers: A Performance Evaluation. *International Journal of Cloud Computing,* 12(4), 512-527.

Kumar, P., & Singh, R. (2021). A Study on Hardware Requirements for Cloud-Based Applications. *International Journal of Cloud Computing and Services Science ,* 10(4), 45-60.

Kumar, V., & Chandra, S. (2023). Human Error in Manual Educational Processes. *Implications for Accuracy and Efficiency. Journal of Educational Research and Review* , 45(3), 87-102.

Kvale, S. (2007). Doing Interviews. Sage.

Lee, E., Park, S., & Kim, D. . (2018). Design Guidelines for User-Friendly Interfaces in Educational Systems. *Journal of Computer Assisted Learning* , 34(5), 587-602.

Lee, T., & Kim, S. (2019). Hardware Considerations for Scalable Web Applications. *Journal of Web Systems and Software Engineering* , 16(3), 77-92.

Li, H., & Wang, Y. (2023). Web-Based Systems in Education:. Modernizing Result Management and Complaint Handling. *Journal of Educational Technology* , 31(2), 78-91.

Li, M., & Tan, A. (2017). "Improving Transparency in Communication Channels for Student Complaints: A Comparative Analysis.*". Journal of Higher Education Policy and Management* , 212-227.

Martins, R. M., & Marques, A. S. (2023). System Integration in Higher Education Institutions: Addressing Communication Gaps. *Journal of Educational Management* , 21(2), 65-80.

Maxwell, J. A. (2022). Designing a Qualitative Study:. A Comprehensive Framework. *Applied Research Journal* , 8(4), 56-73.

Merriam, S. B., & Tisdell, E. J. (2022). Qualitative Research in Education. *A User's Guide. 4th Edition*.

Miles, M. B., Huberman, A. M., & Saldaña, J. (2023). Qualitative Data Analysi. *A Methods Sourcebook* .

Miller, A., & Morgan, K. (2023). Security Challenges in Educational Information Systems. *A Case Study of Higher Education. Journal of Educational Security* , 14(2), 22-38.

Moura, J., De Moura, M. L., & Rocha, H. V. . (2019). Efficiency indicators for educational management in higher education institutions: a systematic literature review. *Quality Assurance in Education*, 27(2), 160-185.

Nelson, D., & Roberts, S. (2023). Scalable and Flexible Systems in Education: Future-Proofing Institutional Technology. *Journal of Educational Technology Development* , 19(3), 101-115.

Nelson, D., & Roberts, S. . (2022). Scalable Educational Systems: Design for Growth. Journal of Educational Technology Development , 19(3), 101-115.

Palinkas, L. A., Horwitz, S. M., Green, C. A., Wisdom, J. P., Duan, N., & Hoagwood, K. (2022). Purposive Sampling Techniques in Educational Settings. *Journal of Research Methodology* , 8(3), 39-55.

Patel, M., & Gupta, A. (2017). The Role of Frameworks in Modern Web Development. *Journal of Computing Technologies*, 37(1), 112-129.

Patel, P., Gupta, S., & Sharma, R. (2020). "Challenges and Opportunities in Implementing Web-Based Systems for Student Result Management and Complaint Resolution: Insights from a Survey Study.". *Computers & Education* , 180-195.

Patel, P., Gupta, S., & Sharma, R. (2020). "Challenges and Opportunities in Implementing Web-Based Systems for Student Result Management and Complaint Resolution: Insights from a Survey Study. *Computers & Education ,* 180-195.

Patel, R., & Singh, M. (2019). The Role of JavaScript in Modern Web Applications: An Analysis. *Journal of Computer Science and Technology ,* 47(2), 184-196.

Patel, S., & Lee, C. (2020). "Data Security Measures in Web-Based Systems: A Systematic Review.". *Journal of Cybersecurity ,* 210-225.

Patel, S., Jones, M., & Garcia, L. (2020). Challenges and Opportunities in Implementing Web-Based Systems for Academic Processes. . *Journal of Information Systems Education ,* 31(2), 103-118.

Patel, S., Jones, M., & Garcia, L. (2020). Challenges and Opportunities in Implementing Web-Based Systems for Academic Processes*. Journal of Information Systems Education,* , 31(2), 103-118.

Patton, M. Q. (2023). . Data Analysis Techniques in Educational Technology Research. *International Journal of Educational Technology* , 21(1), 92-110.

Patton, M. Q. (2023). Data Analysis Techniques in Educational Technology Research. *International Journal of Educational Technology* , 21(1), 92-110.

Preece, J., Rogers, Y., & Sharp, H. (2015). "Interaction Design: Beyond Human-Computer Interaction.". *John Wiley & Sons .*

Qi Yu, Xumin Liu, Athman Bouguettaya & Brahim Medjahed . (04 November 2006). Deploying and managing Web services: issues, solutions, and directions. *The VLDB Journal , pages* 537–572.

Rashid Zafar. (16-17 May 2016). The NoSQL and RDBMS review. *Big Data* .

Roberts, J. A., & Geiger, S. (2023). Communication Challenges in Manual Educational Systems: The Need for Digital Transformation. *Journal of Educational Communication* , 19(3), 112-126.

Security Best Practices in Web Application Development. (2020). Journal of Software Engineering , 55(3), 245258.

Selwyn, N. (2016). "Digital Transformation in Educational Institutions: Challenges and Opportunities.". *Educational Technology Research and Development ,* 1023-1035.

Singh, M., & Agarwal, R. (2017). Evaluating Hardware Performance for Database Management Systems*. Journal of Database Management and Technology* , 12(2), 34-50.

Singh, R., Sharma, P., & Agarwal, K. (2019). The Role of Complaint Management Systems in Enhancing Student Satisfaction*. International Journal of Educational Managemen*t , 33(5), 972-986. vol 5.

Smith, J., & Johnson, A. (2022). "Exploring the Intersection of Technology and Education: A Review of Research Gaps*.". Journal of Educational Technology Research* , 75-92.

Smith, J., & Taylor, P. (2019). Agile Development for Web Applications*. International Journal of Web Development* , 45(2), 123-140.

Smith, J., & Thompson, P. (2023). Enhancing Efficiency Through Automation in Educational Systems. Journal of Education and Technology , 28(3), 98-113.

Smith, J., and Wong, T. (2021). Implementation of Secure Student Record Systems Using Node.js and MongoDB. *Journal of Software Engineering and Application .*

Smith, J., Johnson, A., & Williams, B. (2018). "Enhancing Student Satisfaction through Web-Based Complaint Systems: A Case Study of Higher Education Institutions.". *Journal of Educational Technology & Society* , 112-125.

Smith, J., Johnson, A., & Williams, B. (2019). "Enhancing Transparency and Efficiency in Academic Administration through Web-Based Student Result Management Systems: A Case Study of Implementation in a University Setting.". *Journal of Higher Education Administration* , 112-125.

Smith, R., & Patel, S. (2021). Enhancing Security in Academic Systems: A Case Study on Data Protection. ournal of Information Security in Education , 18(2), 78-92.

Smith, R., & Thompson, A. (2023). The Cost of Time: Analyzing the Inefficiencies of Manual Educational Systems. Journal of Higher Education Management , 145-158.

Smith, R., Brown, K., & Lee, C. (2018). Design and Deployment of a Web-Based System for Student Information Management. International . Journal of Educational Technology, , 12(3), 217-230.

Sommerville, I., & Sawyer, P. . (1997). A Good Practice Guide. John Wiley & Sons. Requirements Engineering .

Taylor, S. J., Bogdan, R., & DeVault, M. L. . (2022). Introduction to Qualitative Research Methods. A Guidebook and Resource. Wiley .

Thompson, E. B., & Garrison, R. D. (2023). Improving Complaint Handling Mechanisms in Educational Institutions. A Review of Best Practices. Journal of Educational Evaluation , 16(3), 78-93.

Wang, H., & Huang, Y. (2021). Enhancing Transparency and CommunicationThe Role of Technology in Educational Administration. Educational Technology Research and Development , 69(4), 835-850.

Wang, Y., & Huang, C. (2021). "Challenges and Solutions in Traditional Result Management Systems: A Case Study in Higher Education Institutions.". International Journal of Educational Management , 530-545.

Wang, Y., & Huang, C. (2021). A Review of Web-Based Systems for Student Result Management and Complaint Resolution in Higher Education Institutions.". International Journal of Educational Technology in Higher Education , 1-20.

Wang, Y., & Johnson, A. (2019). "Ensuring Accessibility in Web-Based Systems: Best Practices and Challenges.". International Journal of Human-Computer Interaction , 320-335.

Wang, Y., & Li, H. (2023). Standardization vs. Autonomy in Educational Institutions: Balancing Uniformity and Flexibility. Journal of Educational Policy and Practice , 28(4), 76-91.

Williams, J., & Thomas, R. (2017). .A Comprehensive Review of JavaScript Frameworks for Web Development. Journal of Web and Software Engineering , 39(4), 255-275.

Williams, L., & Martin, K. (2023). User-Centered Design in Educational Systems: Improving the Student Experience. Journal of User Experience in Education , 17(2), 67-82.

Williams, T., & Martin, R. (2017). User Experience in Educational Platforms: A Student-Centered Approach. Journal of Educational Design , 14(3), 45-60. .

Yang, L., & Liu, Q. (2019). "Addressing Security and Privacy Concerns in Web-Based Student Result Complaint Systems: A Framework for Implementation.". Information Systems Security , 45-58.

Yusof, M., Rosli, A., & Rahim, F. (2021). Web-based Student Result Management System: A Case Study. Journal of Educational Technology , 155-165 vol 2.

Zhang, Y., & Lee, H. (2021). Enhancing Web Application Security with Node.js and Express.js. Journal of Cybersecurity, , 18(1), 105-119.

Appendix A Source Code

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Home Page</title>
<link rel="stylesheet" href="/css/style.css">
<script src="https://kit.fontawesome.com/56adb67a5b.js" crossorigin="anonymous"></script>
<link href="/bootstrap/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
<nav class="navbarnavbar-expand-lg">
<div class="container-fluid">
<imgsrc="/image/ucogo.png" alt="logo" style="width: 50px; height: 50px; border-radius:
50%;">
<ul class="nav justify-content-end">
<li class="nav-item">
<a class="nav-link log-out" href="/registration/page">Register</a>
</li>
```

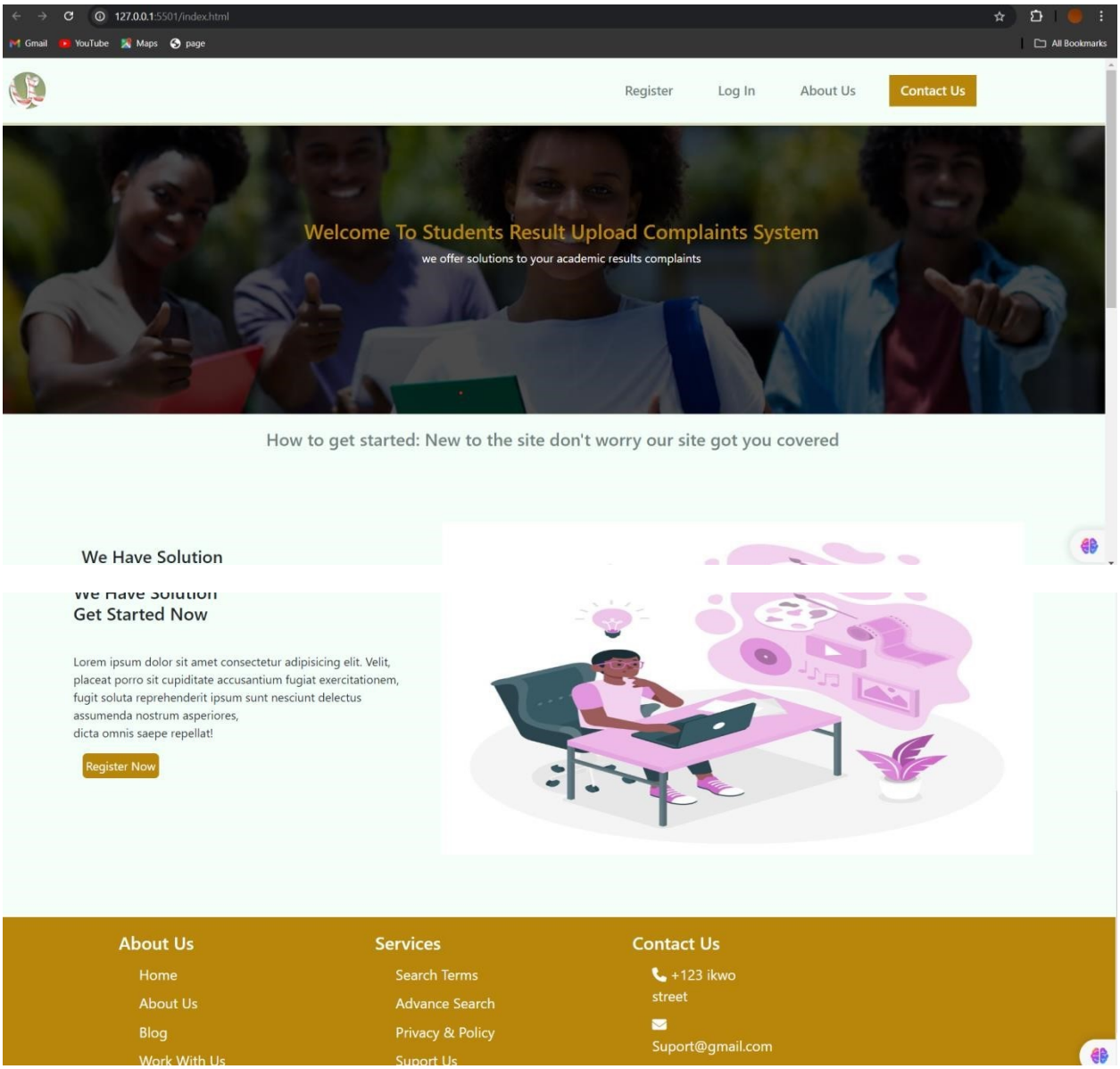
```
<li class="nav-item">
<a class="nav-link log-out" href="/login/page">Log In</a>
</li>
<li class="nav-item">
<a class="nav-link log-out" href="#">About Us</a>
</li>
<li class="nav-item">
<a class="nav-link   contact-nav" href="#" id=" contact-nav"> Contact Us</a> </li>
</ul>
</div>
</nav>
<section class="welcome-msg">
<div class="container">
<div class="txt  container " id="txtwlc">
<h3 id="buy" style="color: #b8860B;">
</h3><span>we offer  solutions to your  academic results  complaints</span><br>
</div>
</section>
<section>
<div class="container">
<div class=" start-txt text-center">
<h4 >How to get started: New to the site  don't worry  our site got you covered</h4>
</div>
</div>
</section>
<div class="wrap-section">
<section class=" wrap-cont">
<div class="headin-text container">
```

```
<h4> We Have Solution <br>
                Get Started Now </h4>
</div>
<div class="container cont">
<div class="text-body">
<p>Loremipsum dolor sit ametconsecteturadipisicingelit. Velit,
        <br>placeatporro sit cupiditateaccusantiumfugiatexercitationem,
<br>fugit solutareprehenderitipsumsuntnesciuntdelectusassumenda nostrum asperiores,<br>
dicta omnissaeperepellat!</p>]
<div class="wrap-btn container">
<button class="btn "> Register Now </button>
</div>
</div>
<img src="/image/creative-thinking-concept-illustration_114360-3507.jpg" alt=""
style="width: 800px; height: 400px;">
</div>
</section>
</div>
<section class="summary">
<div class="container">
<div class="cont">
<div class="about-summary ab-sum">
<h6>About Us</h6>
<ul>
<li><a href="#">Home</a></li>
<li><a href="#"> About Us</a></li>
<li><a href="#">Blog</a></li>
<li><a href="#"> Work With Us</a></li>
<li><a href="#"> Terms &  Conditions</a></li>
</ul>
</div>
<div class="about-summary services">
```

```
<h6>Services</h6>
<ul>
<li><a href="#">Search Terms</a></li>
<li><a href="#"> Advance Search</a></li>
<li><a href="#">Privacy & Policy</a></li>
<li><a href="#">Suport Us</a></li>
<li><a href="#"> Our Stores</a></li>
</ul>
</div>
<div class="about-summary contact">
<h6>Contact Us</h6>
<ul>
<li><i class="fa-solid fa-phone"></i><a href="#">+123 ikwo street</a></li>
<li><i class="fa-solid fa-envelope"></i><a href="#"> Suport@gmail.com</a></li>
<li><a href="#"> +123 customer Care </a></li>
</ul>
</div>
</div>
</div>
</section>
<section  class="footer">
<footer class="copy"> @copy right 2024 </footer>
<div class="footer-icon ">
<i class="fa-brands fa-facebook" id="footer-icon"></i>
<i class="fa-brands fa-twitter" id="footer-icon"></i>
<i class="fa-brands fa-instagram" id="footer-icon"></i>
</div>
</section>
<script>
```

```
    // let buy=document.getElementById("buy")
window.addEventListener("load",function(){ typeWriter()
    }) vari
= 0;
var txt = ' Welcome To  Students Result Upload Complaints System';
var speed = 100; functiontypeWriter() { if (i<txt.length) {
document.getElementById("buy").innerHTML += txt.charAt(i); i++;
setTimeout(typeWriter, speed);
  }
  }
</script>
<scriptsrc="/jquery/dist/jquery.js"></script>
<scriptsrc="/bootstrap/dist/js/bootstrap.bundle.js"></script>
</body>
</html>
```

Appendix B Sample Output



Home Page Sample